
Documentação de Projeto

para o sistema

MobU

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) Joaquim de Moura Thomaz Neto e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo dos professores Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso, Raphael Ramos Dias Costa, orientação acadêmica do professor Cleiton Silva Tavares e orientação de TCC II do professor (a ser definido no próximo semestre).

16/11/2025

Tabela de Conteúdo

Tabela de Conteúdo	ii
Histórico de Revisões	ii
1. Introdução	1
2. Modelo de Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso	4
3. Modelo de Projeto	12
3.1 Diagrama de Classes	12
3.2 Diagramas de Sequência	16
3.3 Diagramas de Comunicação	18
3.4 Arquitetura Lógica: Diagramas de Pacotes	21
3.5 Diagramas de Estados	23
3.6 Diagrama de Componentes	25
4. Projeto de Interface com Usuário	29
4.1 Interfaces Comuns a Todos os Atores	29
4.2 Interfaces Usadas pelo Administrador	30
4.3 Interfaces Usadas pelo Motorista	34
4.4 Interfaces Usadas pelo Passageiro	36
5. Glossário e Modelo de Dados	38
6. Casos de Teste	46
6.1 Testes de Aceitação — Necessidade 1: Passageiro solicitar corrida	47
6.1.2 Necessidade 2 — Motorista aceitar e realizar corrida	48
6.1.3 Necessidade 3 — Pagamento via PIX Off-line	49
6.1.4 Necessidade 4 — Administração de Usuários e Monitoramento do Sistema	50
6.2 Testes de Integração	51
7. Cronograma e Processo de Implementação	57
7.1 Cronograma	57

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Entrega 3	17/09/2025	Início do documento, definindo seções 2.1, 2.2, 2.3 e 4	1.0
Entrega 4	06/10/2025	Inclusão das seções 2.4, 3.1, 3.2 e 3.3	2.0
Entrega 5	31/10/2025	Inclusão das seções 3.4, 3.5, 3.6 e 5	3.0
Entrega 6	12/11/2025	Inclusão da seção 6, 7 e 7.1	4.0
Entrega 7	25/11/2025	Inclusão da seção 6.1, 6.1.2, 6.1.3, 6.1.4 e 6.2	5.0

1. Introdução

A mobilidade urbana em cidades de pequeno e médio porte apresenta desafios relacionados à disponibilidade de motoristas, previsibilidade de preço, segurança no acompanhamento das corridas e organização operacional do serviço. Embora aplicativos de transporte sejam amplamente utilizados em grandes centros, soluções locais costumam apresentar limitações de usabilidade, baixa transparência sobre ganhos e tarifas, pouca integração entre passageiro e motorista e ausência de ferramentas administrativas para monitoramento do negócio. Nesse contexto, identifica-se a necessidade de uma plataforma digital que ofereça uma experiência simples para usuários finais e, ao mesmo tempo, forneça recursos de gestão para a operação do serviço.

Este trabalho apresenta o projeto e o desenvolvimento do MobU, uma solução de mobilidade urbana composta por aplicativo mobile Android, API backend, banco de dados relacional e painel web administrativo. A aplicação desenvolvida permite que passageiros criem conta, solicitem corridas, visualizem estimativas de tarifa, acompanhem o motorista em tempo real e realizem pagamentos em dinheiro ou por Pix off-line. Para motoristas, o sistema oferece cadastro, controle de disponibilidade, aceite e execução de corridas, histórico de viagens e acompanhamento financeiro. Para administradores, o painel web permite gerenciar usuários, motoristas, regiões de operação, relatórios, taxas e informações operacionais da plataforma.

O objetivo geral deste trabalho é especificar, implementar e validar uma plataforma de mobilidade urbana capaz de conectar passageiros, motoristas e administradores em um fluxo integrado de solicitação, execução, pagamento e acompanhamento de corridas. A proposta busca resolver o problema de fragmentação e baixa rastreabilidade observado em soluções locais, oferecendo uma base tecnológica que combine aplicativo móvel, serviços de backend, persistência de dados, integração com mapas, notificações e recursos administrativos.

Como objetivos específicos, o trabalho busca: levantar e documentar os atores, necessidades e histórias de usuário do sistema; modelar os principais casos de uso, fluxos, entidades, componentes e estados da aplicação; projetar interfaces adequadas para passageiros, motoristas e administradores; implementar os módulos essenciais do ecossistema MobU; integrar funcionalidades como autenticação, estimativa de corrida, acompanhamento, Pix off-line, histórico e relatórios; e validar o comportamento do sistema por meio de testes de aceitação, integração, build, lint e execução em emuladores Android.

Este documento está organizado da seguinte forma: a Seção 2 apresenta o modelo de requisitos, incluindo atores, personas, casos de uso e histórias de usuário que fundamentam as funcionalidades do sistema. A Seção 3 descreve o modelo de projeto, contemplando diagramas de classes, sequência, comunicação, arquitetura, estados, componentes e implantação. A Seção 4 apresenta o projeto de interface com o usuário para os diferentes perfis da aplicação. A Seção 5 detalha o glossário e o modelo de dados, incluindo o diagrama entidade-relacionamento. A Seção 6 descreve os casos de teste de aceitação e integração, bem como os resultados obtidos na validação do sistema. Por fim, a Seção 7 apresenta o cronograma e o processo de implementação adotado no desenvolvimento do MobU.

Dessa forma, a documentação consolida a visão funcional, técnica e operacional do sistema desenvolvido, garantindo rastreabilidade entre problema, requisitos, projeto, implementação e testes. Além de servir como registro acadêmico do Trabalho de Conclusão de Curso, o documento também funciona como base de manutenção e evolução futura da plataforma MobU.

O sistema MobU enquadra-se nos objetivos de um Trabalho de Conclusão de Curso em Engenharia de Software por envolver, de forma integrada, atividades de levantamento e análise de requisitos, modelagem de domínio, definição arquitetural, projeto de interface, implementação de componentes de software e validação por meio de testes. A solução desenvolvida não se limita à construção de uma única tela ou funcionalidade isolada, mas compreende um ecossistema composto por aplicativo mobile Android, API backend, banco de dados relacional, painel web administrativo e integrações com serviços externos, como mapas, geolocalização, notificações e pagamento Pix off-line.

Dessa forma, o trabalho contempla práticas fundamentais da Engenharia de Software, como rastreabilidade entre necessidades, requisitos, casos de uso, modelos de projeto, persistência de dados e testes. O desenvolvimento do MobU permite demonstrar a aplicação prática de conceitos estudados ao longo do curso, incluindo arquitetura cliente-servidor, modelagem orientada a objetos, persistência, autenticação, integração entre sistemas, projeto de interfaces e validação funcional.

2. Modelos de Usuário e Requisitos

Esta seção tem como propósito caracterizar os usuários e atores envolvidos no sistema MobU, além de apresentar os requisitos que orientam sua implementação. Para isso, são descritos os principais perfis de usuários, suas necessidades, dores e objetivos, permitindo relacionar o problema identificado às funcionalidades propostas.

A partir dessa caracterização, são apresentados o diagrama de casos de uso e as histórias de usuário, que representam as principais interações esperadas entre passageiros, motoristas, administradores e o sistema. As funcionalidades levantadas foram organizadas de forma a manter rastreabilidade entre necessidades, casos de uso, histórias de usuário e testes, permitindo verificar se cada necessidade relevante do software foi contemplada pelo projeto.

As histórias de usuário foram elaboradas buscando atender às características INVEST, ou seja, devem ser independentes, negociáveis, agregar valor ao usuário, ser estimáveis, sucintas e testáveis. Dessa forma, cada requisito procura representar uma necessidade clara do sistema, evitando ambiguidades e permitindo sua validação por meio dos casos de teste apresentados na Seção 6.

2.1 Descrição de Atores

Passageiro: Usuário que utiliza o aplicativo para solicitar corridas. É responsável por informar origem e destino, acompanhar o trajeto em tempo real, realizar o pagamento e avaliar o motorista ao

final da viagem. Seu principal objetivo é realizar deslocamentos com praticidade, segurança e previsibilidade de custo.

Motorista: Usuário que utiliza o aplicativo para oferecer o serviço de transporte. Recebe solicitações de corridas, navega até o ponto de embarque e conduz o passageiro ao destino informado. Pode consultar seu histórico de corridas e acompanhar seus ganhos. É responsável por manter informações atualizadas no sistema e aceitar ou recusar corridas conforme disponibilidade.

Administrador: Responsável por gerenciar o funcionamento da plataforma por meio de um sistema web. Controla o cadastro de passageiros e motoristas, monitora as corridas realizadas, acompanha indicadores de utilização e gera relatórios financeiros. Também pode configurar taxas da plataforma e atuar em situações que demandem suporte ou resolução de conflitos.

2.2 Modelos de Usuários

Esta subseção tem como objetivo apresentar os modelos de usuários construídos a partir da definição de personas. Para a elaboração dessas personas, foram consideradas as características dos principais atores do sistema como passageiros, motoristas e administradores, bem como suas necessidades, dores e objetivos dentro do contexto da aplicação. As personas a seguir representam perfis típicos dos usuários previstos para a plataforma e servem como referência para orientar decisões de *design*, desenvolvimento e priorização de funcionalidades do sistema.

A Tabela 1 descreve a persona da usuária Mariana Silva, uma passageira que depende do transporte urbano para deslocamentos diários. É possível identificar que, apesar de estar familiarizada com aplicativos de uso comum como *WhatsApp* e *Instagram*, ela sente dificuldades em confiar nos aplicativos locais de transporte por considerá-los confusos e pouco práticos. Também é perceptível que Mariana valoriza a previsibilidade no preço e segurança durante as viagens. Por fim, após análise, observa-se que ela deseja ter maior clareza no custo antes da corrida e a possibilidade de acompanhar o trajeto em tempo real, o que lhe traria mais confiança no serviço.

Mariana Silva	
Descrição	Mariana possui 27 anos, nasceu e cresceu na cidade onde o aplicativo será implantado. Trabalha como recepcionista em uma clínica e depende de transporte diário para se deslocar ao trabalho e para compromissos pessoais. Costuma usar aplicativos como <i>WhatsApp</i> e <i>Instagram</i> , mas nunca utilizou os aplicativos de transporte já existentes na cidade, pois os considera confusos e pouco confiáveis. Busca uma alternativa prática que lhe permita se deslocar com previsibilidade de preço e segurança.

Dores	● Dificuldade em prever o valor de cada corrida com os aplicativos atuais. ● Insegurança por não poder acompanhar a rota em tempo real. ● Pouca confiança em aplicativos locais, pela falta de avaliação de motoristas.
Objetivos	● Ter acesso a corridas rápidas, seguras e acessíveis. ● Saber o valor estimado da corrida antes de embarcar. ● Acompanhar em tempo real o trajeto, sentindo-se mais segura.

Tabela 1. Persona Mariana Silva

A Tabela 2 descreve a persona do usuário Carlos Andrade, um motorista que utiliza o transporte por aplicativo como fonte de renda complementar. Identifica-se que ele já teve experiências anteriores com aplicativos locais, mas considera que eles carecem de recursos de apoio ao motorista e transparência em relação aos ganhos. Nota-se também que Carlos valoriza ferramentas que o ajudam a organizar seu trabalho de forma prática, sem exigir aprendizado complexo de novas tecnologias. Por fim, após análise, percebe-se que ele busca um sistema confiável que facilite o recebimento de corridas e forneça relatórios claros de viagens e rendimentos.

Carlos Andrade	
Descrição	Carlos tem 42 anos e trabalha como motorista de aplicativo em tempo parcial para complementar a renda da família. Já testou os aplicativos locais (LevaAli e MobyGo), mas considera que eles oferecem pouco suporte e não têm transparência nos ganhos. Ele valoriza poder organizar suas corridas e acompanhar de forma simples quanto está ganhando por dia e por semana. Carlos utiliza apenas o necessário em termos de tecnologia, mas aprende rápido quando percebe que o recurso facilita sua rotina.
Dores	● Falta de relatórios claros sobre corridas e ganhos nos aplicativos atuais. ● Necessidade de depender de chamadas aleatórias, sem controle. ● Dificuldade em manter contato com suporte ou resolver problemas por falta de recursos internos no <i>app</i>.
Objetivos	● Receber chamadas de corridas de forma simples e rápida. ● Acompanhar histórico de viagens e rendimento financeiro.

	● Trabalhar em um sistema confiável que ofereça suporte adequado.
--	---

Tabela 2. Persona Carlos Andrade

A Tabela 3 descreve a persona da usuária Fernanda Oliveira, que atua como administradora da plataforma. É possível observar que ela precisa equilibrar a gestão do sistema com suas outras responsabilidades profissionais, o que exige relatórios padronizados e práticos para tomada de decisão. Identifica-se ainda que Fernanda encontra dificuldades em monitorar o desempenho da plataforma com os recursos limitados oferecidos pelos aplicativos concorrentes. Por fim, após análise, verifica-se que sua principal necessidade é dispor de ferramentas que permitam acompanhar a satisfação dos usuários, controlar cadastros e configurar taxas de forma objetiva.

Fernanda Oliveira	
Descrição	Fernanda possui 35 anos, formada em Administração e atua como secretária de gestor da plataforma. Seu papel é garantir que motoristas e passageiros estejam devidamente cadastrados e que o sistema funcione de forma estável. Ela precisa ter acesso a relatórios que mostrem número de corridas realizadas, ganhos, taxas e indicadores de uso do sistema, de modo a tomar decisões sobre melhorias e acompanhar o crescimento do aplicativo.
Dores	● Falta de relatórios padronizados nos aplicativos locais. ● Dificuldade em monitorar qualidade do serviço (avaliações e <i>feedbacks</i>). ● Necessidade de ajustar taxas e políticas de forma manual.
Objetivos	● Ter acesso a relatórios claros e detalhados sobre o funcionamento do sistema. ● Acompanhar o desempenho de motoristas e satisfação de passageiros. ● Configurar taxas e gerenciar usuários de forma prática.

Tabela 3. Persona Fernanda Oliveira

2.3 Modelo de Casos de Uso e Histórias de Usuários

Esta subseção apresenta os casos de uso e as histórias de usuário que orientam o desenvolvimento do sistema MobU. O diagrama de casos de uso representa as principais interações entre passageiros,

motoristas, administradores e serviços externos, evidenciando como cada ator participa dos fluxos essenciais da aplicação.

As histórias de usuário complementam essa visão ao descrever, em linguagem simples e orientada a valor, as funcionalidades esperadas para cada perfil. Elas permitem relacionar as necessidades levantadas no Documento de Visão com funcionalidades implementáveis e testáveis, favorecendo a priorização, a estimativa de esforço e a validação do sistema.

2.3.1 Diagrama de Casos de Uso

A Figura 1 apresenta o diagrama de casos de uso do sistema proposto. No diagrama, são representados três atores principais: Passageiro, Motorista e Administrador, que interagem diretamente com o sistema. Além deles, são exibidos dois sistemas externos: o Serviço de Mapas e Geolocalização, utilizado para cálculo de rotas e estimativas de tempo, e o *Gateway* de Pagamentos(Futuramente). O passageiro acessa o sistema para criar conta, solicitar corridas, acompanhar o trajeto, efetuar pagamento e avaliar o motorista. O Motorista utiliza o sistema para receber corridas, aceitar ou recusar solicitações, navegar até o passageiro e ao destino, marcar etapas da viagem e consultar relatórios de ganhos. O Administrador gerencia usuários, motoristas, taxas, regiões de operação, relatórios do sistema e solicitações de suporte. Ela representa visualmente essas interações, bem como as relações entre os casos de uso e os sistemas externos.

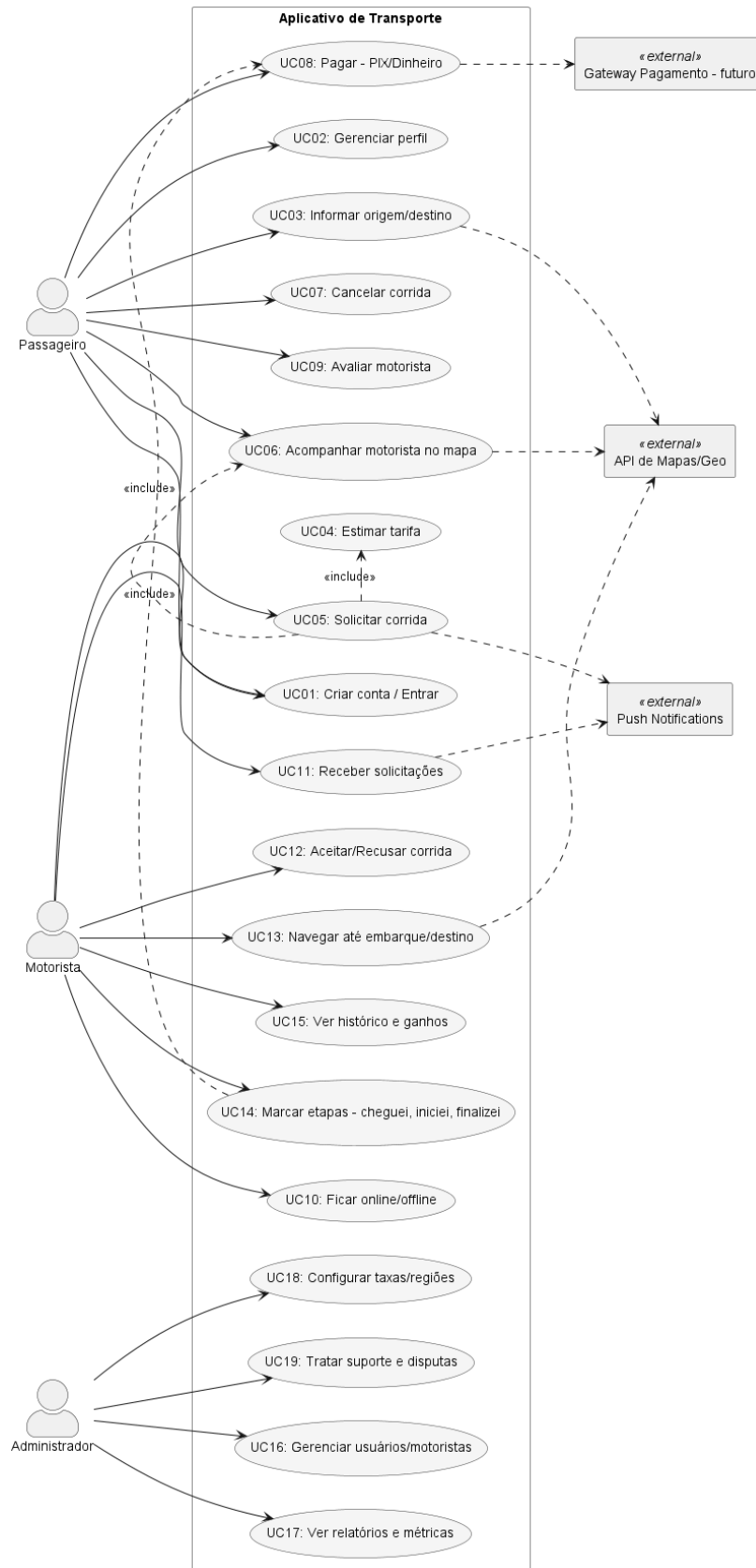


Figura 1 – Diagrama de Casos de Uso

2.3.2 Histórias de Usuário

Nesta seção, são listadas as histórias de usuário levantadas para o sistema proposto. Para fins de organização, utiliza-se identificadores no formato *US#ID*, em que *US* se refere a *User Story*.

As histórias de usuário identificadas para o sistema são:

US1. Como passageiro, eu gostaria de criar uma conta e gerenciar meu perfil para que eu possa acessar o sistema e utilizar seus serviços.

US2. Como passageiro, eu gostaria de informar origem e destino para que eu possa solicitar uma corrida.

US3. Como passageiro, eu gostaria de ver a estimativa de tarifa e tempo de chegada do motorista para decidir se quero confirmar a corrida.

US4. Como passageiro, eu gostaria de acompanhar a rota do motorista em tempo real para saber onde ele está.

US5. Como passageiro, eu gostaria de efetuar o pagamento via PIX ou em dinheiro para concluir a corrida com segurança.

US6. Como passageiro, eu gostaria de avaliar o motorista após a corrida para contribuir com a reputação da plataforma.

US7. Como motorista, eu gostaria de entrar no sistema e definir meu *status* como *online* ou *offline* para indicar quando estou disponível para receber corridas.

US8. Como motorista, eu gostaria de receber solicitações de corrida com informações de origem, destino e valor estimado para decidir se aceito a viagem.

US9. Como motorista, eu gostaria de navegar até o passageiro e depois até o destino usando o mapa do *app* para otimizar a rota.

US10. Como motorista, eu gostaria de marcar as etapas da corrida (cheguei, iniciei, finalizei) para registrar corretamente o andamento da viagem.

US11. Como motorista, eu gostaria de acessar um painel com meus ganhos diários e semanais para acompanhar meus resultados.

US12. Como administrador, eu gostaria de gerenciar contas de passageiros e motoristas para manter o sistema organizado e seguro.

US13. Como administrador, eu gostaria de acompanhar relatórios de corridas e métricas do sistema para tomar decisões estratégicas.

US14. Como administrador, eu gostaria de configurar taxas e regiões de operação para ajustar o funcionamento da plataforma conforme a cidade.

US15. Como administrador, eu gostaria de visualizar e responder solicitações de suporte ou disputas para resolver problemas de forma rápida.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Nesta seção, são apresentados os diagramas de sequência do sistema, assim como seus Contratos de Operações. O objetivo destes diagramas é descrever os fluxos de interação existentes entre os usuários e o sistema, representando a troca de mensagens e as dependências entre módulos da aplicação.

A Figura 2 representa o diagrama de sequência do sistema relacionado ao fluxo de solicitação de corrida. Nesse fluxo, o Passageiro informa sua origem e destino e, ao confirmar o pedido, o sistema consulta o Serviço de Mapas para calcular a rota e o tempo estimado.

Após essa etapa, a aplicação envia uma notificação em tempo real ao Motorista disponível mais próximo, que poderá aceitar ou recusar a corrida.

Esse diagrama está diretamente relacionado aos casos de uso UC03 (Informar origem/destino), UC04 (Estimar tarifa), UC05 (Solicitar corrida) e UC11 (Receber solicitação).

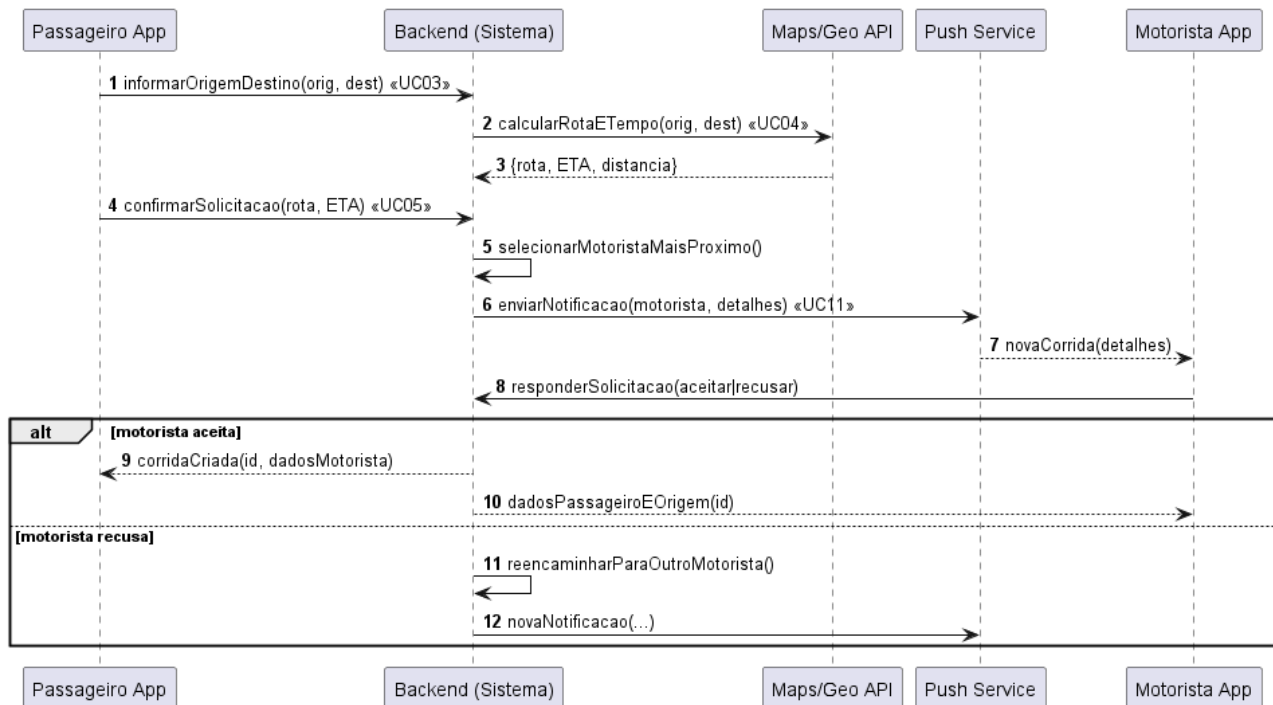


Figura 2 – Diagrama de Sequência Solicitar Corrida e Notificar Motorista

Contrato	Solicitar corrida e notificar motorista
Operação	<i>requestRide(origin: String, destination: String)</i>
Referências cruzadas	UC03 Informar origem/destino, UC04 Estimar tarifa, UC05 Solicitar corrida, UC11 Receber solicitação
Pré-condições	O passageiro deve estar autenticado no sistema e com geolocalização ativa
Pós-condições	O motorista mais próximo é notificado e a corrida é registrada no banco de dados como “pendente”

Tabela 4. Contrato Solicitar Corrida e Notificar Motorista

A Figura 3 mostra o diagrama de sequência referente ao fluxo de pagamento da corrida. Após o término da viagem, o sistema apresenta as opções de PIX ou Dinheiro. No processo de pagamento em dinheiro ou pix, o sistema atualiza o status localmente. Esse fluxo está relacionado aos casos de uso UC08 (Efetuar pagamento) e UC14 (Finalizar corrida).

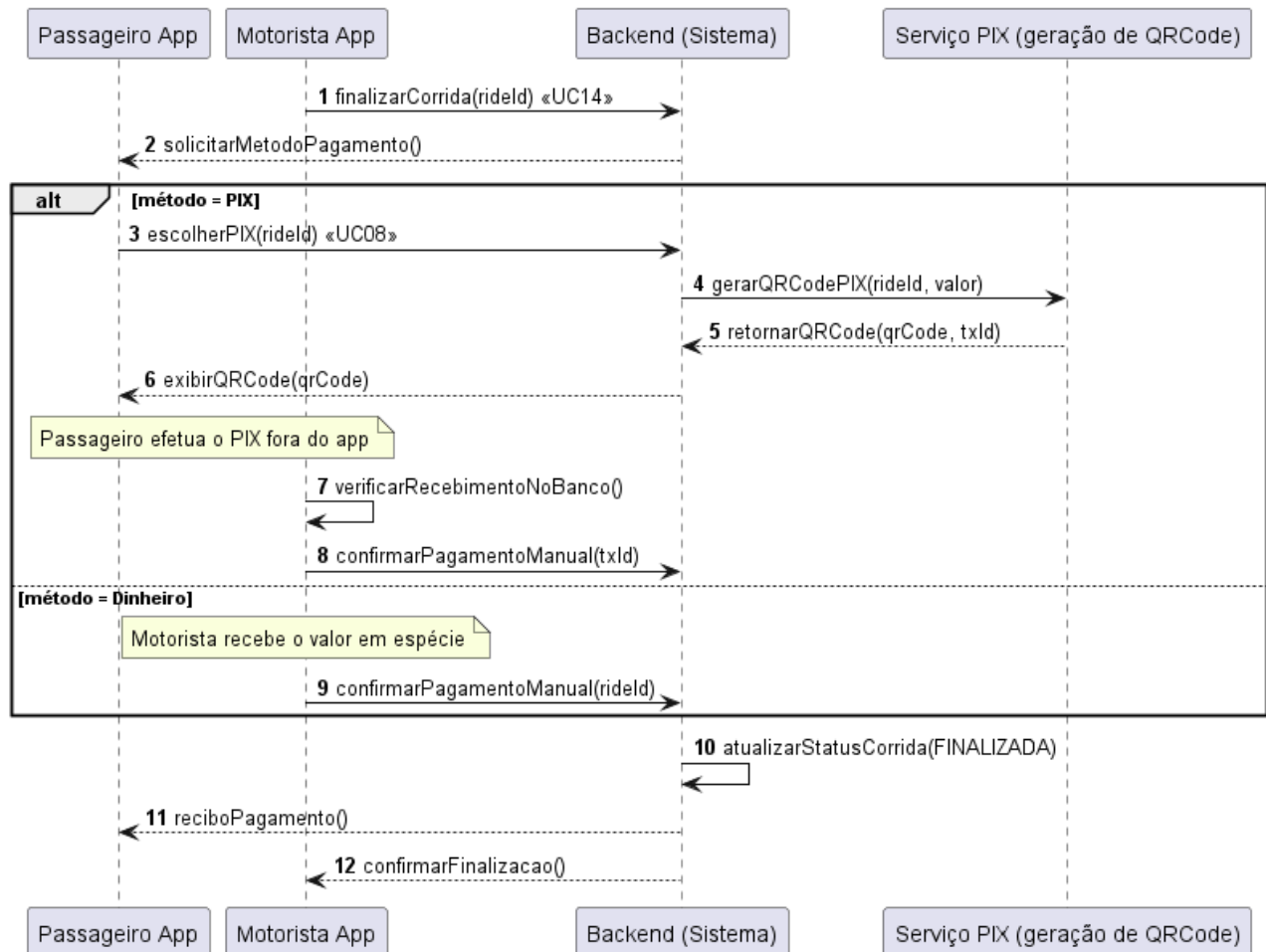


Figura 3 – Diagrama de Sequência Efetuar Pagamento

Contrato	Efetuar pagamento da corrida
Operação	<i>processPayment(paymentMethod: Enum, rideId: Int)</i>

Referências cruzadas	UC08 Efetuar pagamento, UC14 Finalizar corrida
Pré-condições	A corrida deve ter sido concluída e estar aguardando pagamento
Pós-condições	O pagamento é confirmado (PIX) ou marcado como concluído (dinheiro), alterando o status da corrida para “finalizada”

Tabela 5. Contrato Efetuar Pagamento da Corrida

A Figura 4 representa o diagrama de sequência do fluxo de avaliação do motorista. Após o encerramento da corrida e o processamento do pagamento, o Passageiro pode registrar uma avaliação que inclui nota e comentário. O sistema valida os dados e salva a avaliação, atualizando a média do motorista no banco de dados. Esse fluxo está associado ao caso de uso UC09 (Avaliar motorista).

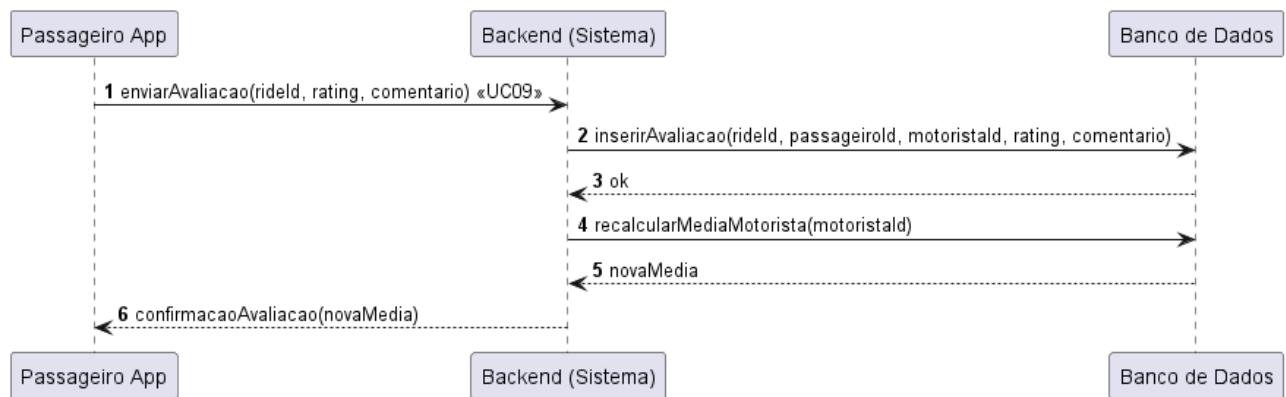


Figura 4 – Diagrama de Sequência Avaliação do Motorista

Contrato	Registrar avaliação do motorista
Operação	<i>submitReview(rideId: Int, rating: Int, comment: String)</i>
Referências cruzadas	UC09 Avaliar motorista
Pré-condições	A corrida deve ter sido finalizada e o pagamento concluído
Pós-condições	A avaliação é registrada e a nota média do motorista é atualizada no sistema

Tabela 6. Contrato Registrar Avaliação do Motorista

A Figura 5 ilustra o diagrama de sequência do painel administrativo, que demonstra como o Administrador acessa o sistema web, solicita relatórios e recebe informações agregadas de corridas, motoristas e usuários. O fluxo inclui a requisição dos dados, o processamento interno e a resposta exibida em forma de gráficos e métricas. Esse diagrama se relaciona aos casos de uso UC16 (Gerenciar usuários/motoristas), UC17 (Ver relatórios e métricas) e UC18 (Configurar taxas/regiões).

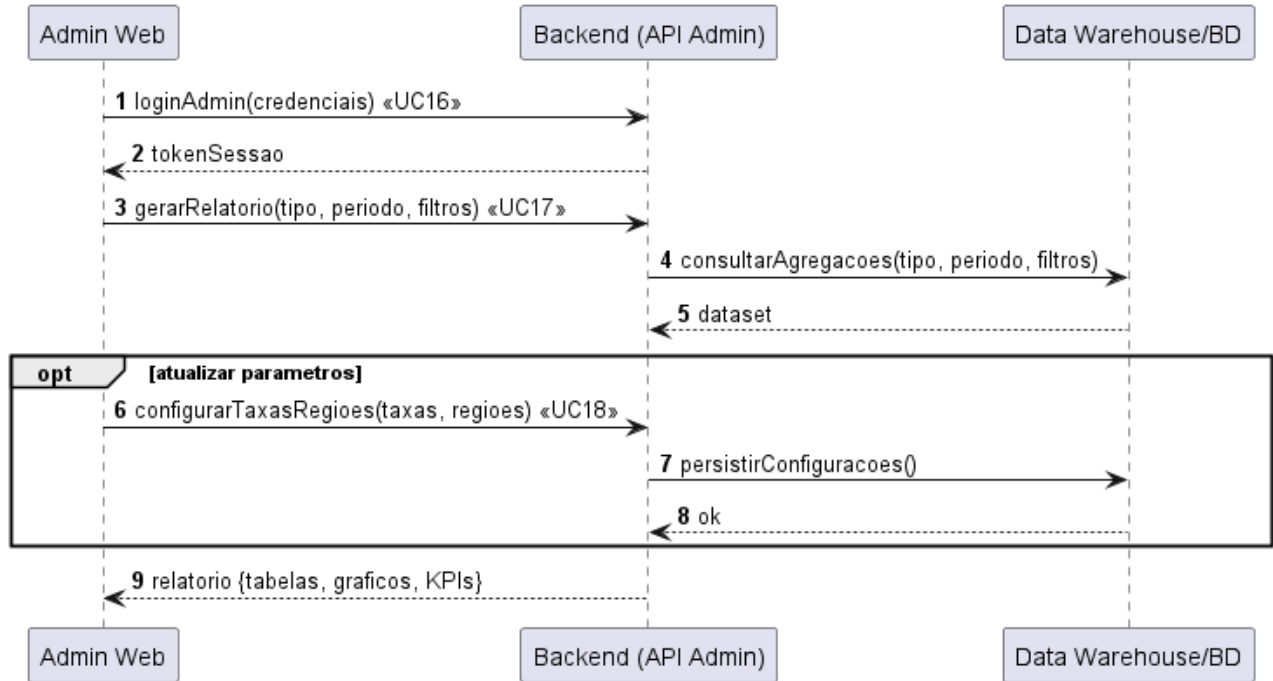


Figura 5 – Diagrama de Sequência Gerar Relatórios Administrativos

Contrato	Gerar relatórios administrativos
Operação	<i>generateReport(type: Enum, dateRange: String)</i>
Referências cruzadas	UC16 Gerenciar usuários, UC17 Ver relatórios, UC18 Configurar taxas
Pré-condições	O administrador deve estar autenticado no painel web
Pós-condições	O relatório solicitado é gerado e apresentado na interface em formato gráfico

Tabela 7. Contrato Gerar Relatórios Administrativos

3. Modelos de Projeto

Esta seção apresenta os modelos de projeto utilizados para detalhar a estrutura e o comportamento do sistema MobU. Os diagramas descritos permitem transformar os requisitos levantados anteriormente em uma visão técnica da solução, apoiando a implementação dos principais fluxos da aplicação.

São apresentados modelos estruturais e comportamentais, incluindo diagrama de classes, diagramas de sequência, diagramas de comunicação, arquitetura lógica, diagrama de estados, diagrama de componentes e modelo de implantação. Esses modelos foram elaborados de forma coerente com os casos de uso e com o modelo de dados, permitindo compreender como as entidades, componentes e usuários interagem para realizar as funcionalidades propostas.

3.1 Diagrama de Classes

O diagrama de classes representa a estrutura principal do domínio do sistema MobU, contemplando as entidades responsáveis pelos fluxos de cadastro, solicitação de corrida, execução da viagem, pagamento, avaliação e administração. A modelagem considera os principais atores do sistema, como passageiro, motorista e administrador, além das entidades necessárias para registrar corridas, pagamentos, regiões, históricos e informações operacionais.

A entidade relacionada à corrida ocupa papel central no modelo, pois conecta passageiro, motorista, localização, status da viagem e dados de pagamento. Essa estrutura permite representar adequadamente os casos de uso levantados e mantém coerência com o Diagrama de Entidade e Relacionamento apresentado posteriormente. Dessa forma, o diagrama de classes favorece a implementação do sistema e contribui para a manutenção e evolução da solução.

A Figura 6 ilustra o diagrama de classes do pacote *controllers*, evidenciando as dependências diretas entre os controladores e os serviços correspondentes que contêm as regras de negócio. É possível observar a relação entre *PassageiroController* e *CorridaService*, *MotoristaController* e *RelatorioService*, entre outras.

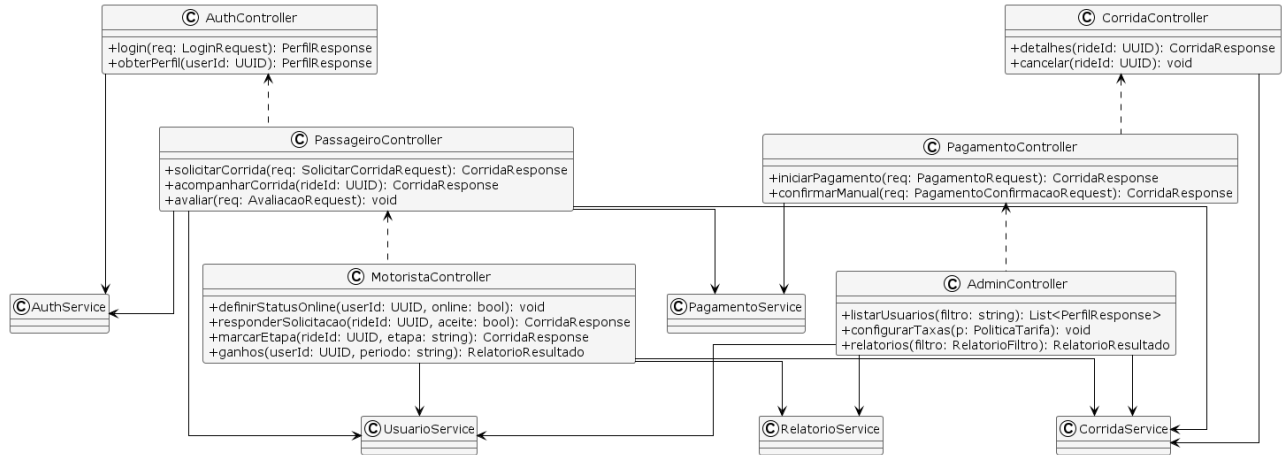


Figura 6 – Diagrama de Classes: Controllers

O diagrama de classes apresentado a seguir representa o pacote de *services*, responsável pela camada de regras de negócio do aplicativo.

Essas classes implementam as operações centrais da aplicação, como o gerenciamento de corridas, cálculo de tarifas, pareamento entre motoristas e passageiros, processamento de pagamentos e geração de relatórios.

O pacote inclui serviços de autenticação (*AuthService*), gestão de usuários (*UsuarioService*), cálculo de preço dinâmico (*PrecificacaoService*), correspondência de motoristas (*MatchingService*), gerenciamento de *status* (*MotoristaStatusService*), controle de corridas (*CorridaService*), pagamentos (*PagamentoService*) e notificações (*NotificacaoService*).

Há ainda o *RelatorioService*, que fornece métricas de desempenho e estatísticas operacionais tanto para motoristas quanto para administradores.

As Figuras 7 e 8 apresentam o diagrama de classes dos *services*, mostrando as dependências com os repositórios de persistência e com integrações externas como *APIs* de mapas e serviços de *push*.

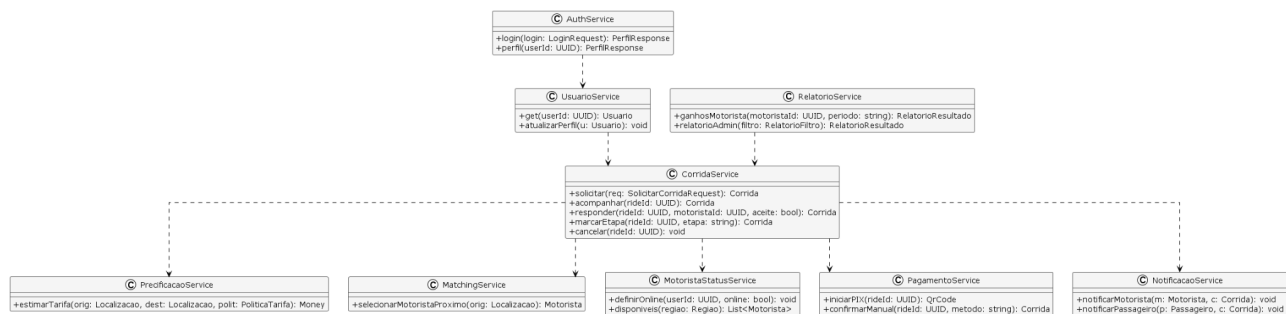


Figura 7 – Diagrama de Classes: Services01



Figura 8 – Diagrama de Classes: Services02

O diagrama de classes a seguir descreve o pacote domain, que contém os modelos de dados fundamentais do sistema. Essas classes representam as entidades centrais que sustentam o funcionamento do aplicativo, como Usuario, Passageiro, Motorista, Veiculo, Corrida, Pagamento e Avaliacao.

Cada entidade foi estruturada com seus respectivos atributos e relacionamentos, garantindo coerência com os processos do negócio. Por exemplo, a classe Corrida mantém vínculos diretos com Passageiro, Motorista e Rota, além de conter informações de status, valores e horários. A classe Pagamento está associada à Corrida, representando tanto pagamentos via PIX quanto em dinheiro, ambos processados manualmente no aplicativo. Além disso, o modelo inclui entidades auxiliares como Localizacao, PoliticaTarifa, Regiao e *Money*, usadas para padronizar endereços, políticas de preço e valores monetários.

A Figura 9 apresenta o diagrama de classes do domínio, permitindo visualizar de forma clara as relações entre as entidades e seus papéis dentro do fluxo geral da aplicação.

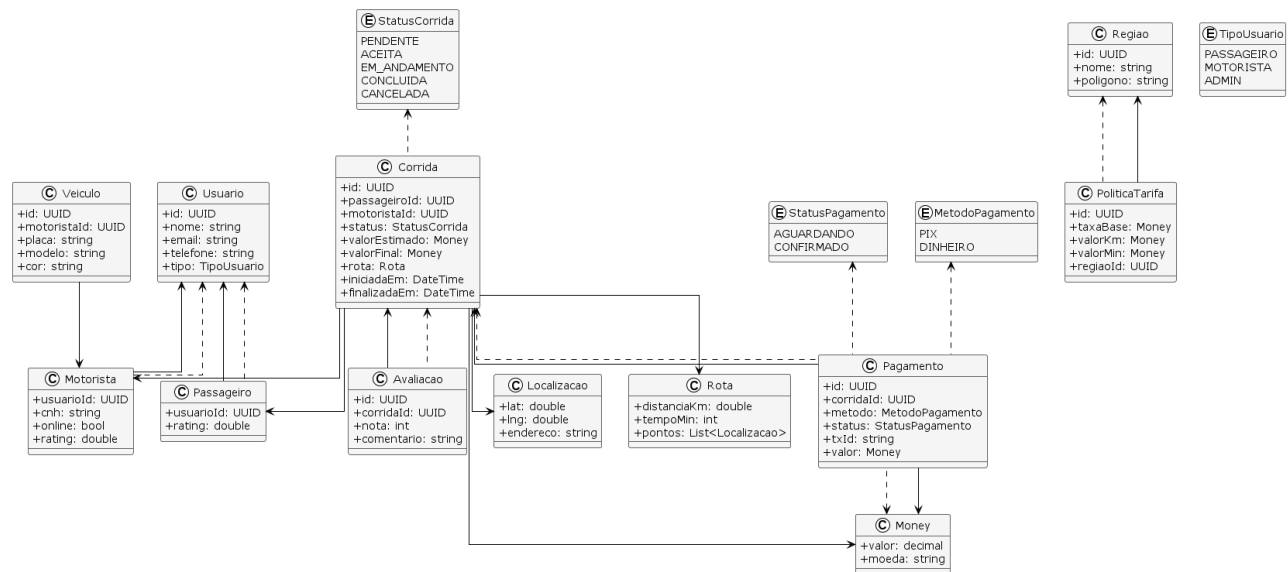


Figura 9 – Diagrama de Classes: Domain / Models

O diagrama de classes a seguir representa o pacote de *repositories*, responsável pela camada de persistência de dados. Essas classes são definidas como interfaces, pois a implementação pode variar de acordo com a tecnologia de banco de dados adotada, mantendo assim o princípio da abstração e separação de responsabilidades.

Cada repositório é responsável por manipular uma entidade específica, garantindo a persistência e recuperação de informações essenciais ao funcionamento do sistema. Entre os repositórios estão *UsuarioRepository*, *CorridaRepository*, *PagamentoRepository* e *AvaliacaoRepository*, que lidam com as operações de CRUD e consultas filtradas.

A Figura 10 exibe o diagrama de classes de *repositories*, demonstrando a estrutura modular de acesso aos dados e sua conexão com a camada de *services*.

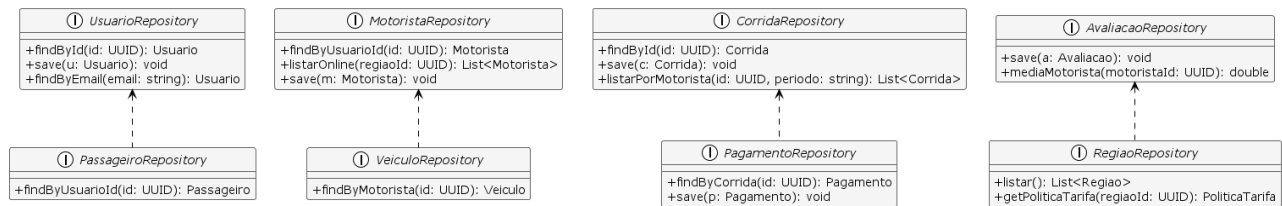


Figura 10 – Diagrama de Classes: Repositories

Por fim, a figura a seguir apresenta o diagrama de classes referente aos *Data Transfer Objects* (DTOs), que têm como função padronizar a comunicação entre o cliente (aplicativo móvel) e o servidor (API). Esses objetos simplificam o envio e recebimento de dados, evitando o tráfego de entidades completas e protegendo a estrutura interna do sistema.

Entre os principais DTOs estão *LoginRequest*, *PerfilResponse*, *SolicitarCorridaRequest*, *CorridaResponse*, *PagamentoRequest*, *PagamentoConfirmacaoRequest* e *AvaliacaoRequest*. Também foram definidos objetos voltados à geração de relatórios, como *RelatorioFiltro* e *RelatorioResultado*, utilizados por administradores e motoristas para visualizar métricas de desempenho.

A Figura 11 apresenta o diagrama de classes dos DTOs, mostrando a utilização de tipos de domínio como *Localizacao*, *Rota* e *Money* para garantir consistência entre os dados de transporte e pagamento.

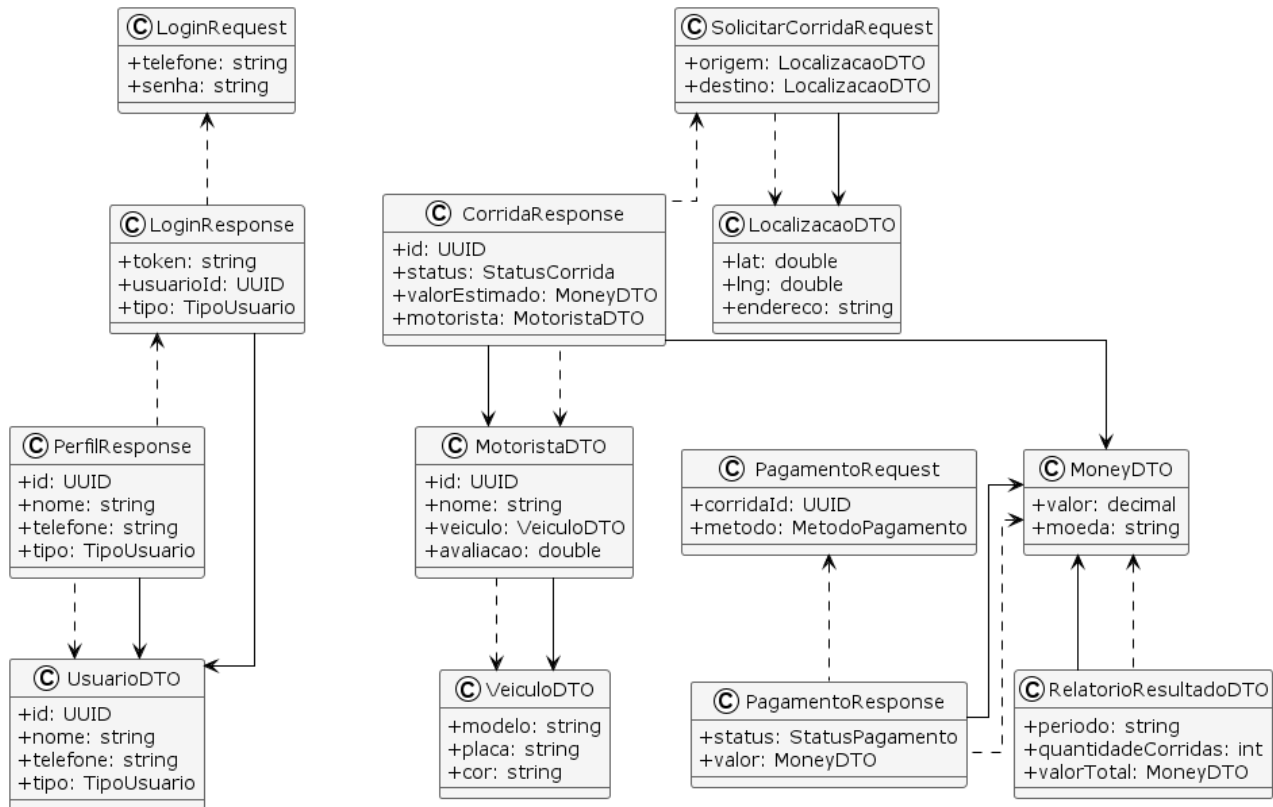


Figura 11 – Diagrama de Classes: DTOs

3.2 Diagramas de Sequência

Nesta seção, são apresentados os diagramas de sequência que modelam os fluxos principais do sistema de transporte proposto. Esses diagramas têm como objetivo mapear os caminhos críticos de uso da aplicação, evidenciando quem se comunica com quem, quais mensagens são trocadas e em que ordem.

Foram selecionados quatro fluxos representativos: solicitação de corrida, pagamento (PIX/dinheiro, confirmação manual), avaliação do motorista e relatórios administrativos. Juntos, eles cobrem as interações essenciais entre Passageiro, Motorista e Administrador, além das integrações necessárias com serviços externos.

A Figura 12 representa o fluxo de solicitação de corrida. O Passageiro informa origem e destino; o sistema consulta o serviço de mapas para obter rota, distância e tempo estimado; em seguida, identifica o motorista mais próximo e envia uma notificação de corrida. O motorista pode aceitar ou recusar. Em caso de recusa, a solicitação é reencaminhada a outro motorista. Este fluxo está relacionado aos casos de uso UC03 (Informar origem/destino), UC04 (Estimar tarifa), UC05 (Solicitar corrida) e UC11 (Receber solicitação).

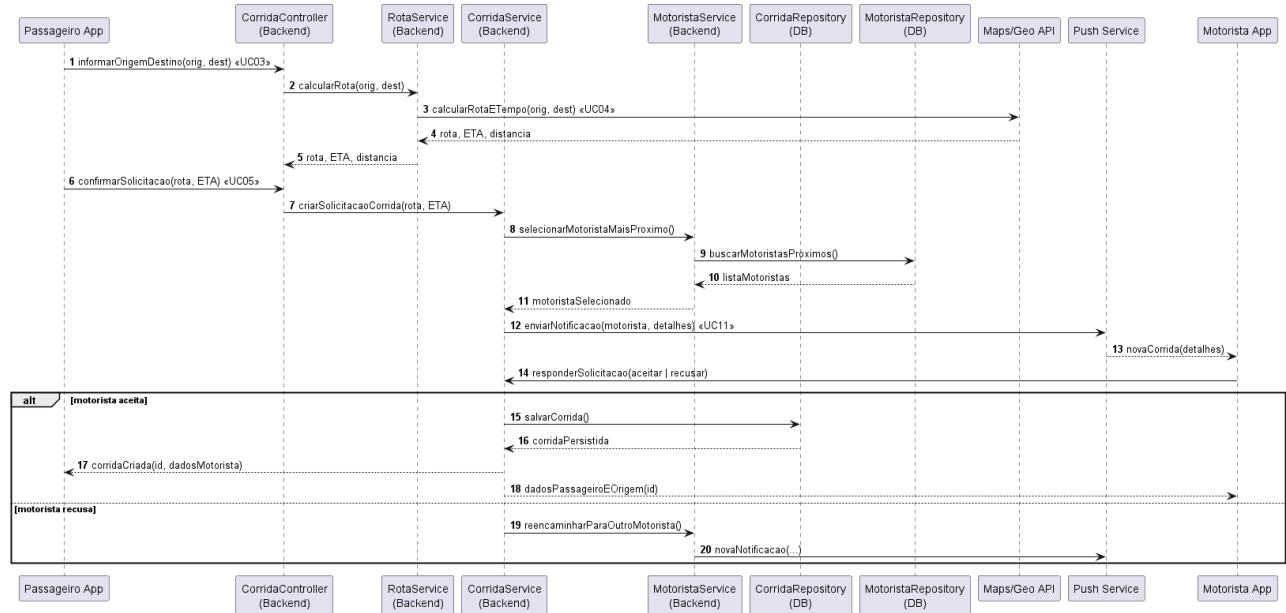


Figura 12 – Diagrama de Sequência Solicitação de Corrida

A Figura 13 descreve o encerramento da corrida e o pagamento. O sistema apresenta os métodos PIX e Dinheiro. Para PIX, o *backend* gera um *QR Code* (sem integração bancária automática); o motorista verifica manualmente no app do banco se o valor entrou e confirma no sistema. Para Dinheiro, o motorista confirma manualmente o recebimento. Em ambos os casos, a corrida passa a Finalizada, e recibos são disponibilizados. Este fluxo cobre UC08 (Efetuar pagamento) e UC14 (Finalizar corrida).

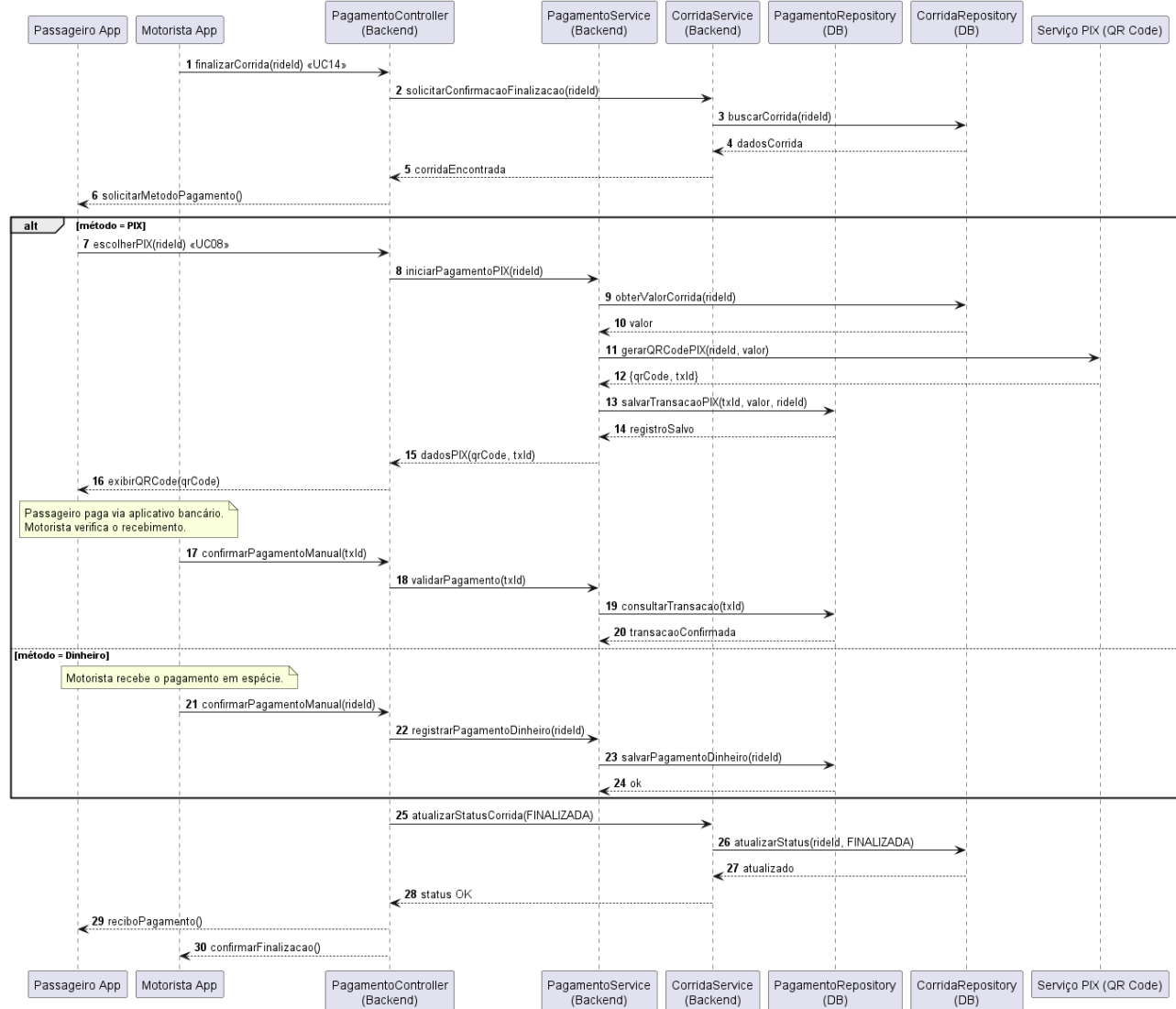


Figura 13 – Diagrama de Sequência Encerramento da Corrida e Pagamento

A Figura 14 mostra o fluxo de avaliação do motorista pelo Passageiro após o pagamento. O cliente informa nota e comentário; o sistema armazena a avaliação e recalcula a média do motorista. Esse processo retroalimenta a qualidade do serviço e influencia futuras seleções de motoristas. Relaciona-se ao caso de uso UC09 (Avaliar motorista).

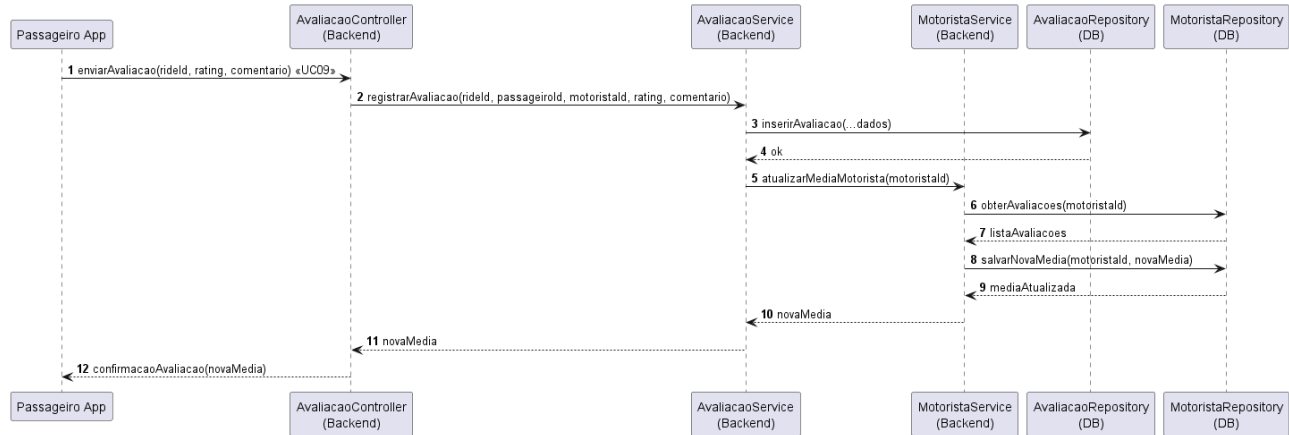


Figura 14 – Diagrama de Sequência Avaliação do Motorista

A Figura 15 ilustra o painel administrativo. O Administrador realiza login, solicita relatórios (por período, região, motorista etc.), e o *backend* consulta o repositório/*warehouse* para agregar dados de corridas e pagamentos. Opcionalmente, o Administrador pode ajustar configurações (taxas, regiões), que são persistidas para uso nos cálculos. Relaciona-se aos casos de uso UC16 (Gerenciar usuários/motoristas), UC17 (Ver relatórios e métricas) e UC18 (Configurar taxas/regiões).

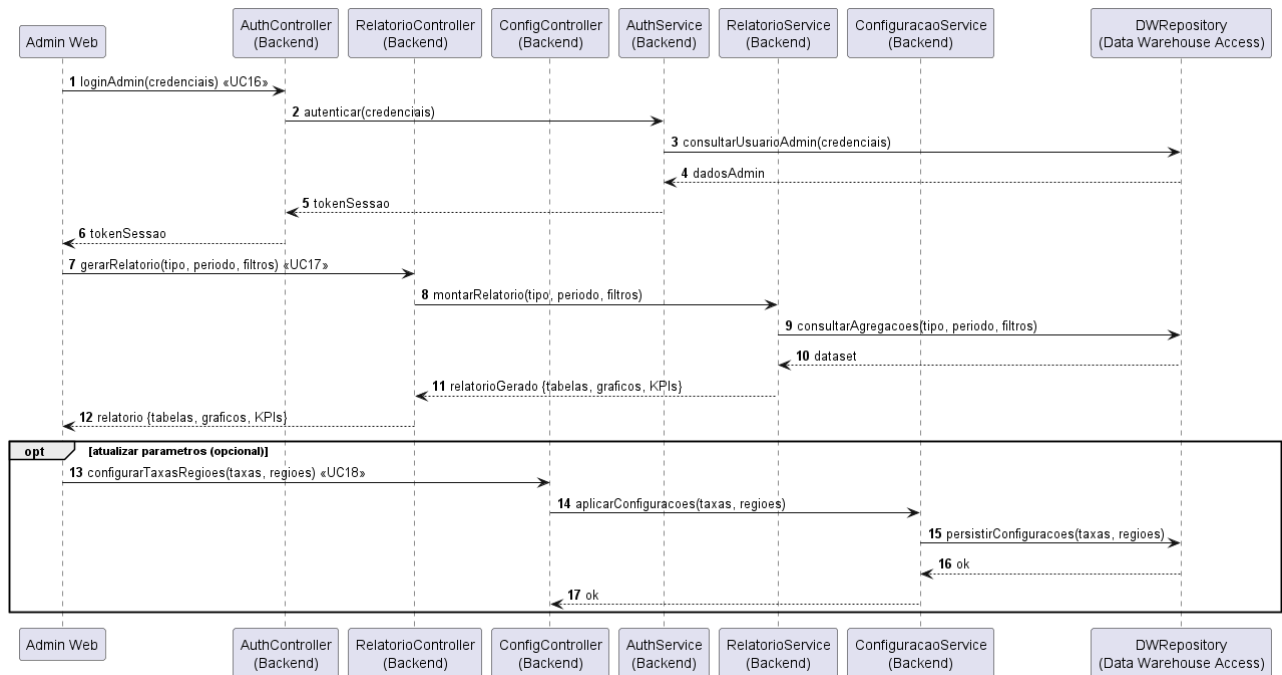


Figura 15 – Diagrama de Sequência Painel Administrativo

3.3 Diagramas de Comunicação

Nesta seção, são apresentados os diagramas de comunicação que modelam as trocas de mensagens entre atores, camadas e serviços do sistema de transporte. O objetivo é documentar quem se

comunica com quem e em que ordem, cobrindo os principais fluxos: autenticação, solicitação e despacho de corrida, atualização de *status*, pagamento e relatórios administrativos. Os diagramas a seguir complementam os diagramas de sequência, servindo como registro das interfaces e contratos de mensagem utilizados na aplicação.

A Figura 16 apresenta as trocas de mensagens para autenticação de usuários (passageiros e motoristas). O aplicativo envia as credenciais para a *API*, que delega a validação ao *AuthService*. Em seguida, os dados do usuário são obtidos no *UsuarioRepository* e um *token* de sessão é retornado ao cliente. Esse fluxo garante a segurança de acesso e inicializa o contexto de sessão para as demais operações do sistema.

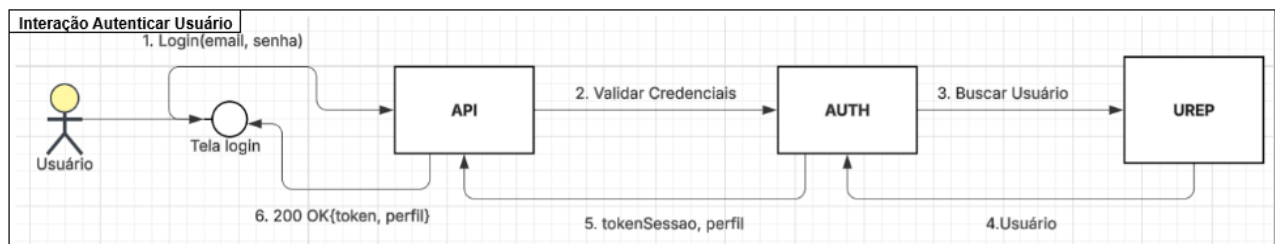


Figura 16 – Diagrama de Comunicação Autenticar Usuário

A Figura 17 descreve o fluxo de comunicação para solicitação de corrida. Após informar origem/destino, a *API* consulta a *Maps/Geo API* para estimativa de rota/tempo, chama o *MatchingService* para selecionar o motorista disponível mais próximo e utiliza o *PushService* para notificar o motorista. A resposta do motorista (aceite/recusa) retorna à *API*, que confirma ao passageiro e publica o estado inicial da corrida.

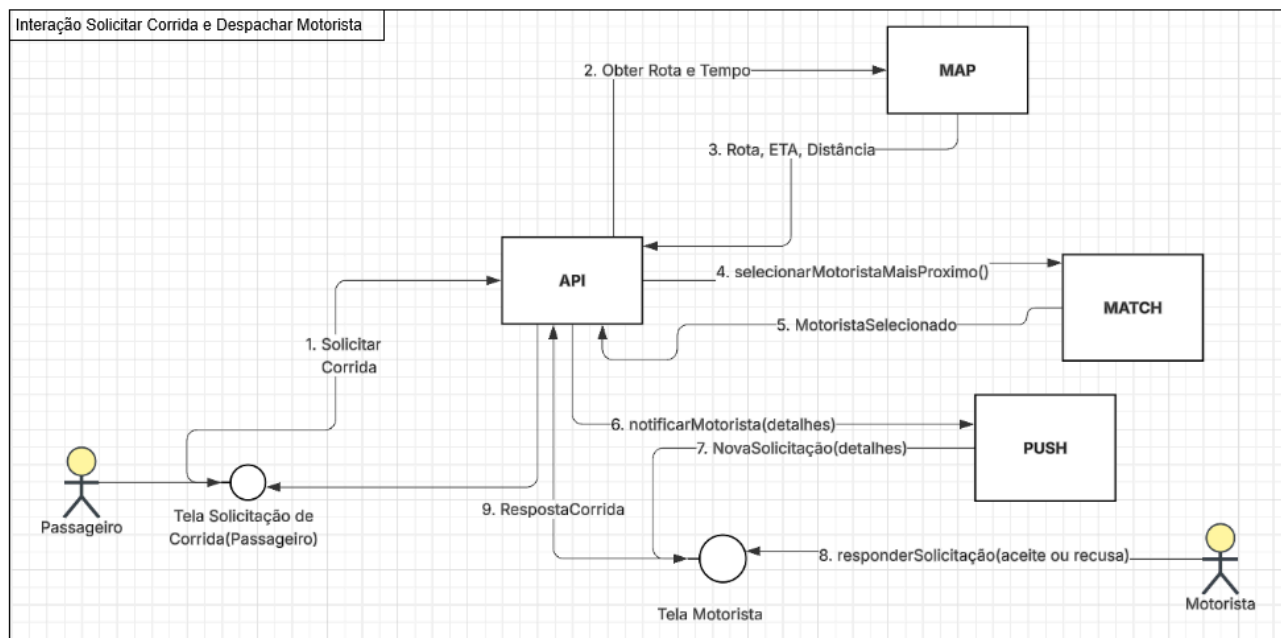


Figura 17 – Diagrama de Comunicação Solicitar Corrida e Despachar Motorista

A Figura 18 mostra as mensagens trocadas durante a execução da corrida. O motorista envia eventos de *status* (a caminho, chegou, iniciou, finalizou) à *API*. A *API* persiste no *CorridaRepository*, recalcula *ETA* quando necessário via *Maps/Geo API* e notifica passageiro/motorista com atualizações relevantes. Esse fluxo mantém ambas as partes informadas e cria o histórico operacional da corrida.

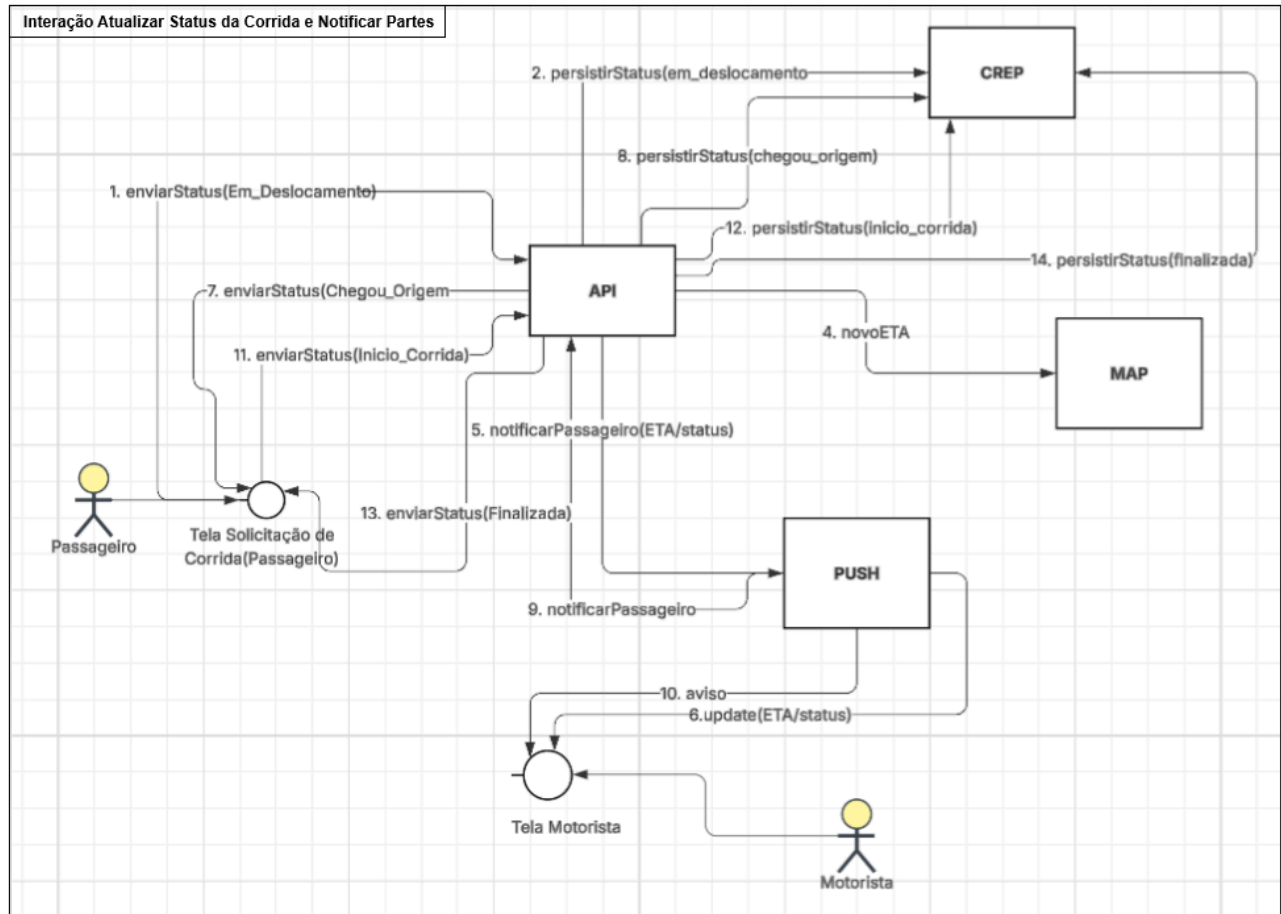


Figura 18 – Diagrama de Comunicação Atualizar Status da Corrida e Notificar Partes

A Figura 19 detalha as comunicações no pagamento. Ao finalizar a corrida, o passageiro escolhe PIX ou dinheiro. Para PIX, a API solicita ao *PixQrService* a geração do *QR Code* e o exibe no app; o motorista confere manualmente no seu banco e confirma na *API*. Para dinheiro, o motorista apenas confirma o recebimento. Em ambos os casos, a *API* atualiza a Corrida para “Finalizada” e emite recibos.

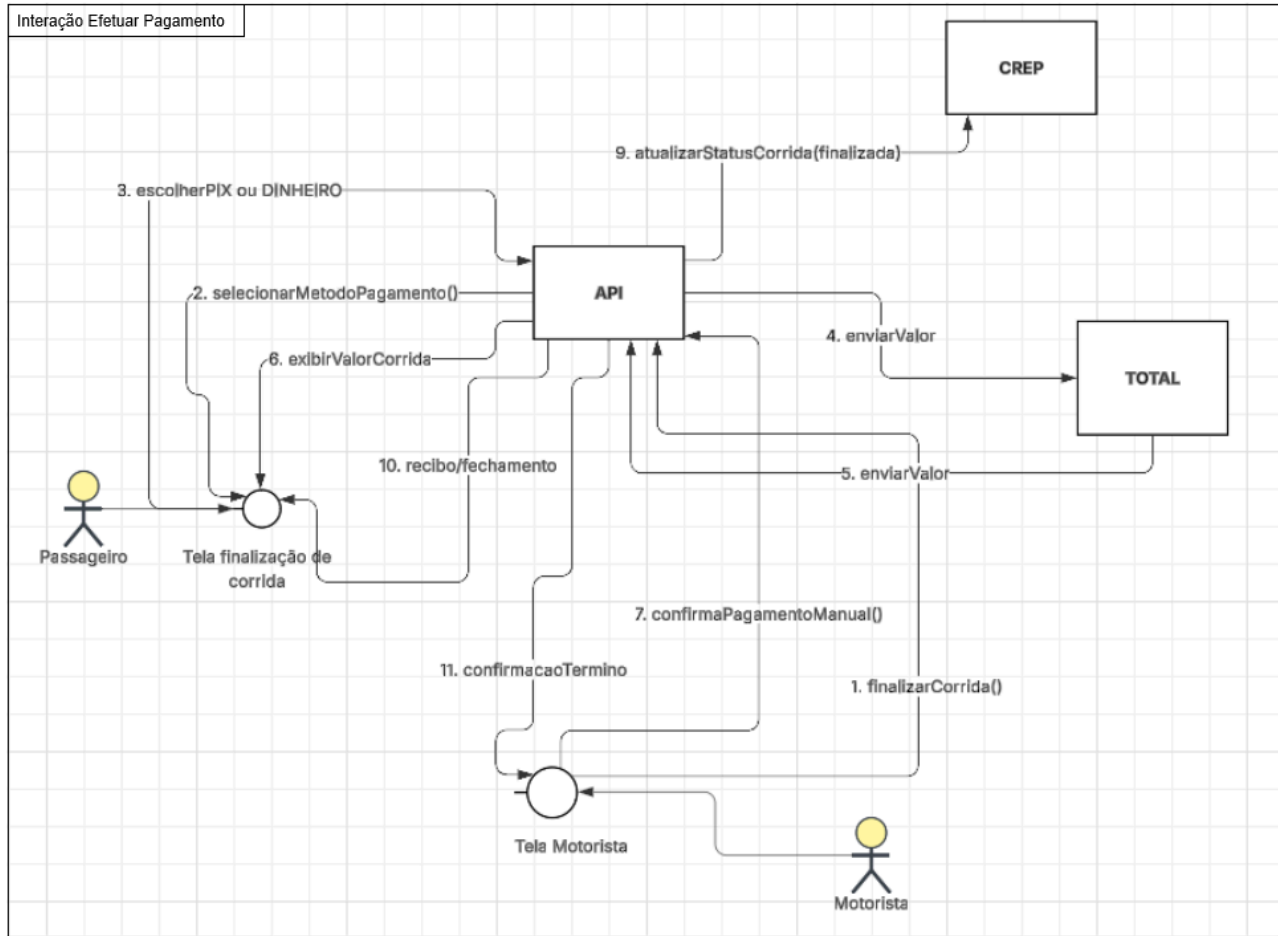


Figura 19 – Diagrama de Comunicação Efetuar Pagamento

3.4 Arquitetura

A Figura 20 representa o diagrama de pacotes, também conhecido como diagrama de arquitetura, do sistema MobU. Nele é possível observar a organização estrutural dos principais módulos da aplicação, representando a separação de responsabilidades entre as camadas que compõem o sistema.

O primeiro pacote representa a aplicação mobile, que contém a Interface de Usuário (UI, do inglês *User Interface*), responsável pela interação direta com os diferentes perfis de usuários — passageiro, motorista e administrador. Essa camada é responsável por exibir as informações e capturar as ações realizadas pelos usuários durante o uso do aplicativo.

A aplicação mobile se comunica com a MobU API, que corresponde à camada intermediária do sistema e é responsável por processar as regras de negócio e gerenciar as comunicações com os serviços externos.

Dentro desta camada, estão definidos os pacotes Controllers, Services, Jobs e Utils, os quais implementam as funcionalidades principais da aplicação.

- O pacote Controllers contém os pontos de entrada da API e é responsável por intermediar as requisições entre o cliente e o backend.
- O pacote Services contém a lógica de negócio central do sistema, como o gerenciamento de corridas, pagamentos e autenticações.
- O pacote Jobs trata de tarefas assíncronas e agendadas, como envio de notificações ou processamento de relatórios.
- O pacote Utils contém funções auxiliares reutilizáveis em diferentes partes da aplicação.
- O pacote Libs reúne bibliotecas e adaptadores compartilhados entre serviços.

Por fim, a camada Data é responsável pelo acesso e persistência dos dados, representando a comunicação com o banco de dados e os modelos de entidades da aplicação. Ela contém o pacote Models, que define as estruturas de dados principais do sistema, como *Usuário*, *Corrida*, *Pagamento* e *Avaliação*.

Essa arquitetura segue o padrão camada UI → API (business) → Data, proporcionando alta coesão e baixo acoplamento entre os módulos, o que facilita a manutenção, a escalabilidade e futuras expansões da aplicação.

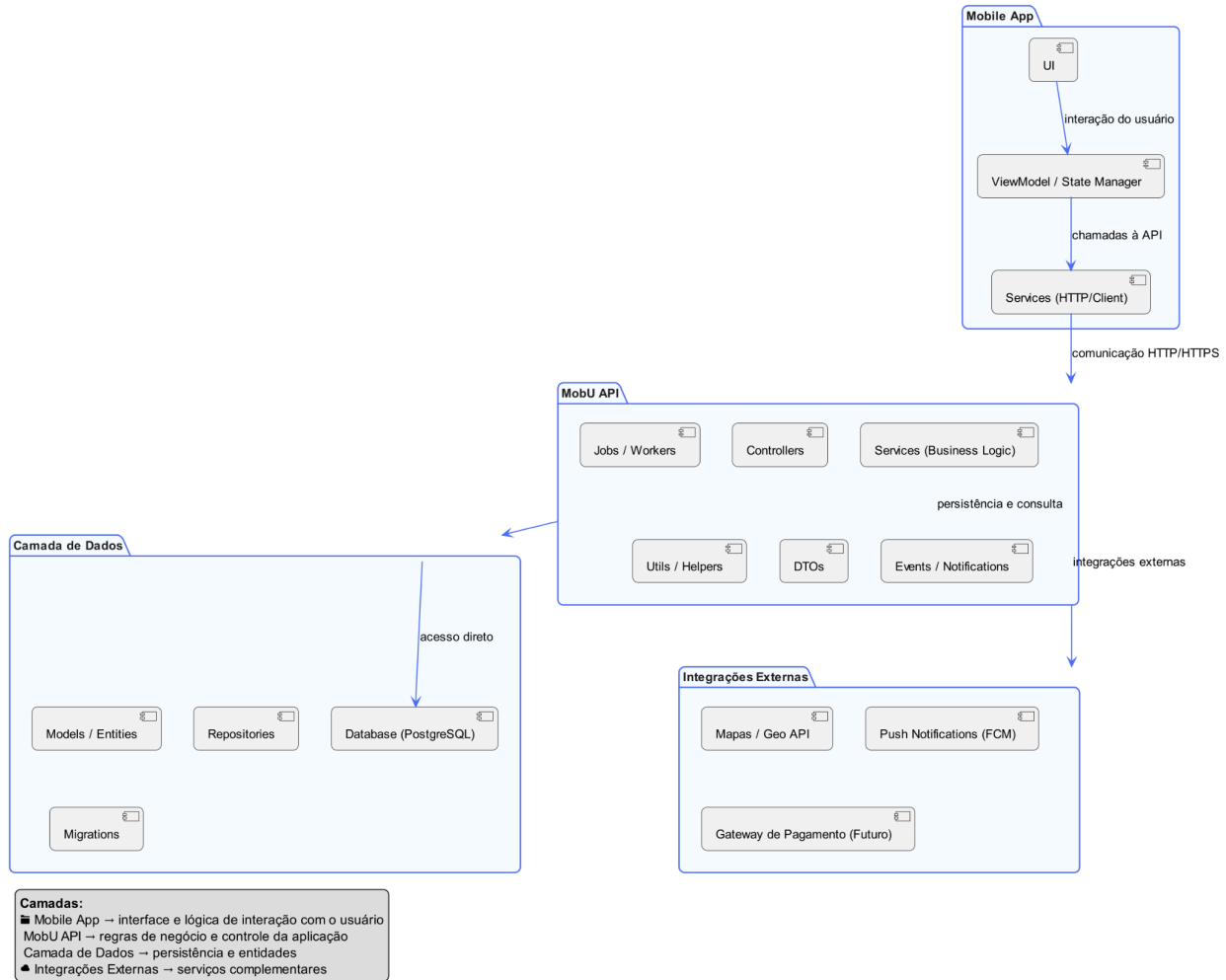


Figura 20 – Diagrama de Pacotes

3.5 Diagramas de Estados

O diagrama representado pela Figura 21 contém a lógica de mudança de estados pelos quais uma corrida passa no sistema MobU. É possível observar que a corrida percorre uma sequência de estados que refletem as diferentes fases do processo de transporte, desde sua solicitação até a finalização ou cancelamento.

Inicialmente, o estado Solicitada representa a criação da corrida pelo passageiro. Em seguida, a corrida pode ser Despachada, momento em que o sistema tenta associar um motorista disponível à solicitação. A partir desse ponto, o motorista pode aceitar ou recusar a corrida, resultando respectivamente nos estados Aceita ou Recusada — neste último caso, o sistema pode reencaminhar a solicitação para outro motorista disponível.

Quando aceita, a corrida avança para o estado A Caminho da Origem, no qual o motorista se dirige até o ponto de embarque do passageiro. Ao chegar, a corrida transita para o estado Chegou à Origem, e, após o embarque, é iniciada, entrando no estado Em Andamento.

Por fim, o estado Finalizada indica a conclusão da corrida, com o registro das informações de trajeto e valor.

O diagrama também contempla transições de cancelamento, que podem ocorrer em diferentes fases do processo, seja por parte do passageiro, do motorista ou por algum motivo forçado pelo sistema. Essas transições levam a corrida ao estado Cancelada, que, assim como o estado final de Finalizada, encerra o ciclo de vida da corrida dentro do sistema.

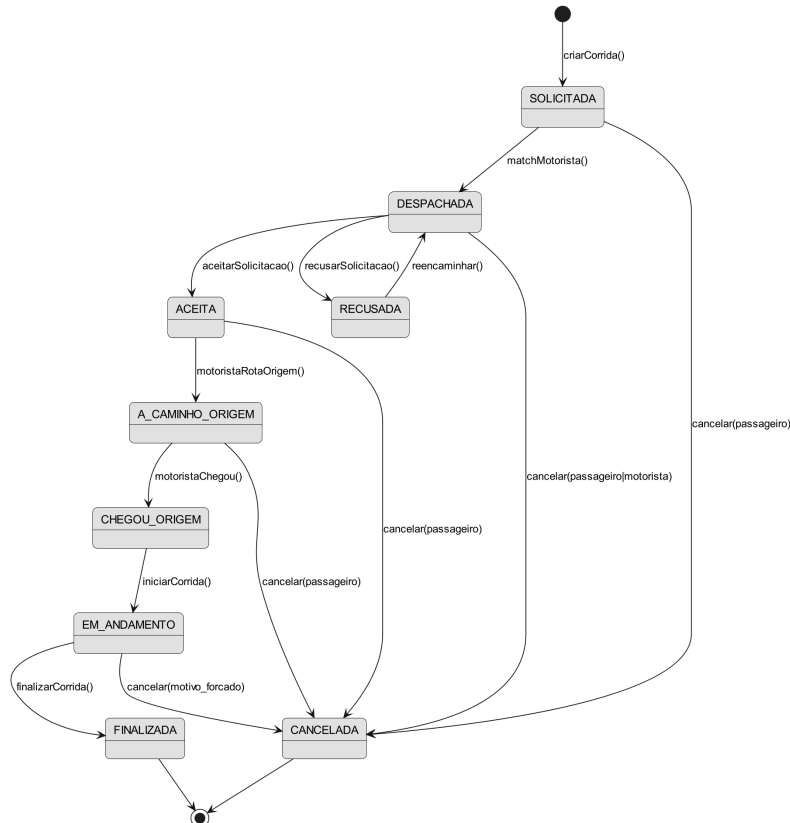


Figura 21 – Diagrama de Estados da Corrida

A Figura 22 representa o Diagrama de Estados do Pagamento no sistema MobU. Esse diagrama descreve o ciclo de vida de um pagamento associado a uma corrida, desde o momento em que a corrida é finalizada até a confirmação ou cancelamento da transação.

O processo se inicia no estado Aguardando, quando a corrida é concluída e o usuário deve escolher o método de pagamento. A partir desse ponto, existem dois fluxos possíveis:

1. Caso o usuário escolha PIX, o sistema gera um QR Code, e o pagamento entra no estado PIX Gerado.
2. Caso o usuário opte por dinheiro, o pagamento segue para o estado Aguardando Confirmação do Motorista, onde o motorista deve confirmar o recebimento.

A partir do estado PIX Gerado, o pagamento pode ser Confirmado após a validação do txId ou pode Expirar caso o QR Code não seja utilizado dentro do prazo estabelecido.

Além disso, o pagamento pode ser Cancelado manualmente em qualquer um desses estados intermediários.

Por fim, os estados Confirmado, Expirado e Cancelado representam os estados finais do processo de pagamento, encerrando o ciclo de transações para aquela corrida específica.

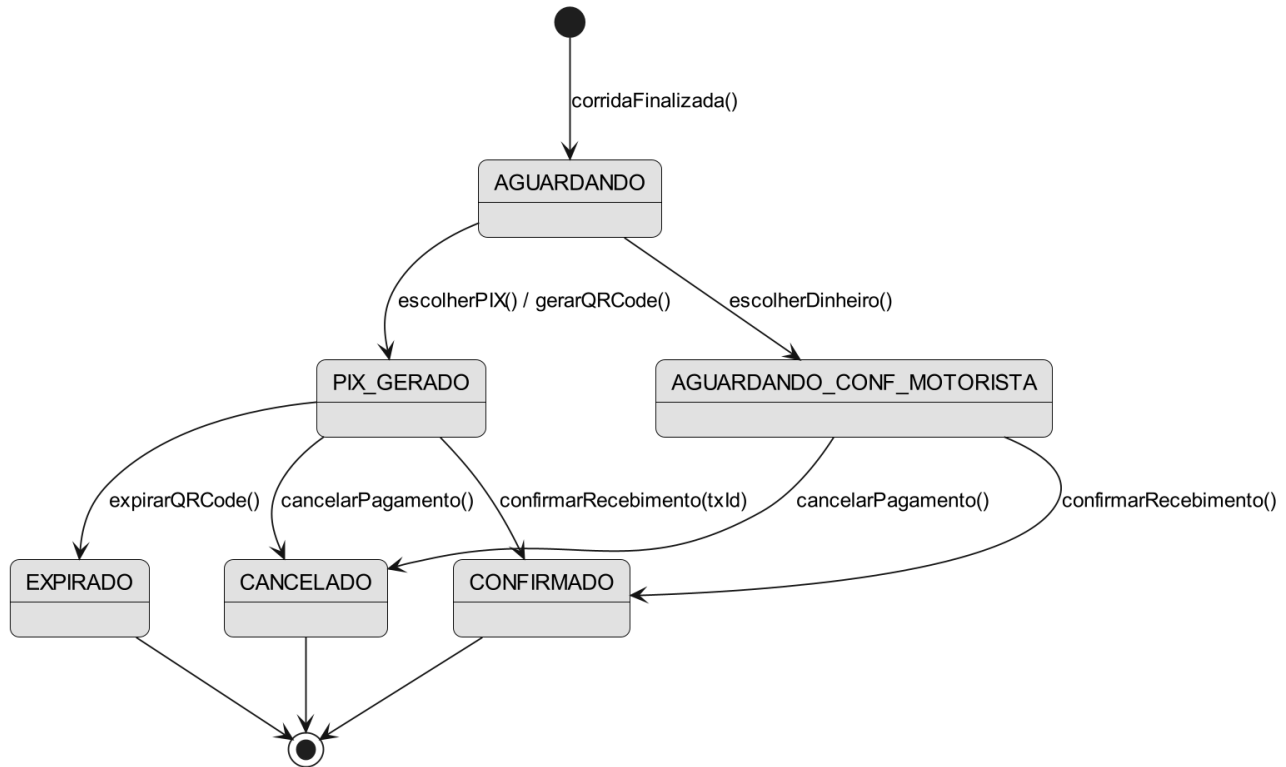


Figura 22 – Diagrama de Estados do Pagamento

3.6 Diagrama de Componentes e Implantação.

O diagrama representado pela Figura 23 mostra a organização dos principais componentes que compõem a arquitetura lógica do sistema *MobU*. Nele é possível observar dois grandes módulos: o Mobile App e a MobU API, além de seus elementos de integração com serviços externos e infraestrutura de persistência de dados.

O Mobile App representa a aplicação móvel executada em dispositivos Android e iOS, composta pela camada de Interface de Usuário (UI), responsável pelas páginas, componentes visuais e estados da aplicação. Essa camada se comunica com a MobU API por meio de requisições HTTP/HTTPS, realizadas através do módulo Services, que encapsula as funções de acesso à API e controle de fluxo de dados entre o cliente e o servidor.

A MobU API representa o núcleo de processamento do sistema, responsável por tratar as regras de negócio e integrar as diferentes partes da aplicação.

Dentro dessa camada, encontram-se pacotes especializados:

- Controllers, que atuam como pontos de entrada das requisições;
- Services, que implementam a lógica de negócio principal;
- Jobs, responsáveis por tarefas assíncronas e agendadas;
- Utils e Libs, que contêm funções e bibliotecas auxiliares;
- Models, que representam as entidades e dados do sistema;
- APIs, dedicadas às integrações externas, e Events, que tratam notificações e mensagens internas.

O Message Queue representa o serviço de mensageria responsável por filas de eventos, enquanto o SGBD (PostgreSQL) é responsável pelo armazenamento persistente de informações. Além disso, o sistema se integra a APIs externas, como Mapas/Geo, Push Notifications (FCM) e o Gateway de Pagamento (futuro), garantindo comunicação com serviços complementares e expansibilidade futura.

Essa estrutura modular facilita a manutenção, o escalonamento e a reutilização de componentes, promovendo um acoplamento reduzido e alta coesão entre as camadas.

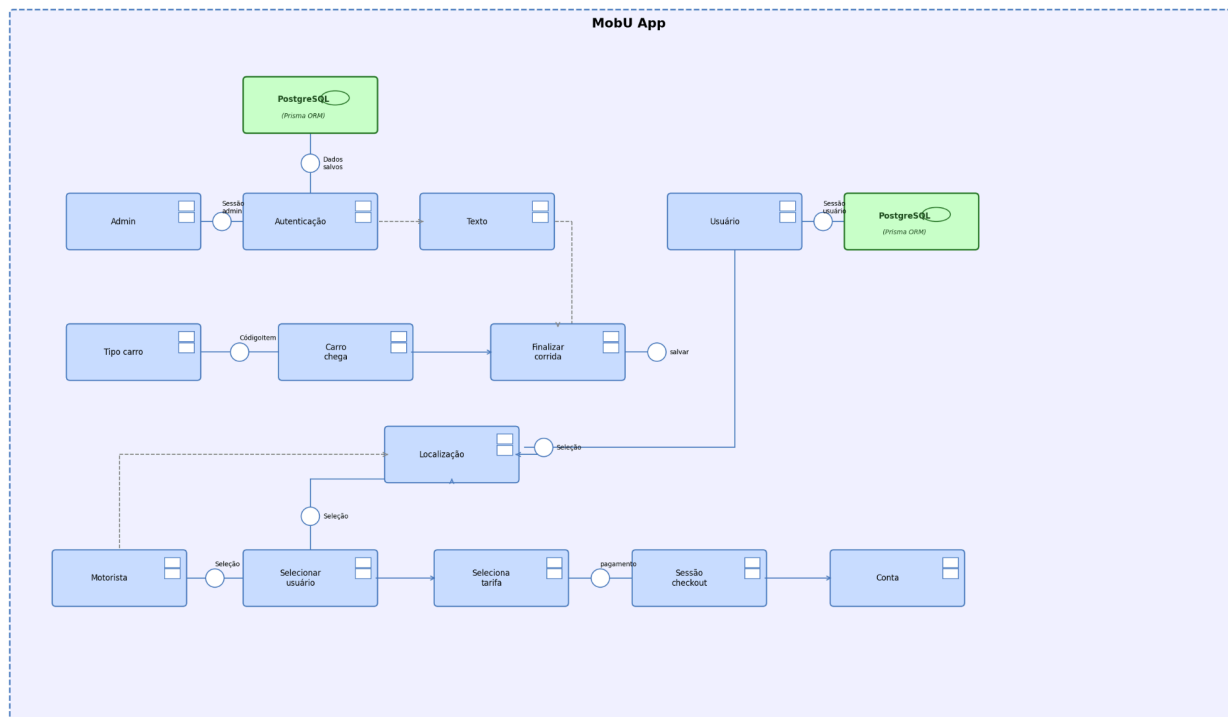


Figura 23 – Diagrama de Componentes da Aplicação

A Figura 24 apresenta o Diagrama de Componentes da aplicação MobU, evidenciando os principais módulos do sistema, suas responsabilidades e as interações estabelecidas entre eles. O diagrama permite compreender a organização lógica da aplicação, bem como o fluxo de comunicação entre os componentes responsáveis pelas funcionalidades centrais do sistema.

No centro da arquitetura está o componente MobU App, que representa o aplicativo móvel utilizado pelos usuários finais. Esse componente concentra a interface de uso e coordena a interação com os demais módulos do sistema, atuando como ponto principal de acesso às funcionalidades da aplicação.

O sistema contempla diferentes perfis de acesso, representados pelos componentes Usuário, Motorista e Admin. Cada um desses módulos possui responsabilidades específicas e se comunica com o componente de Autenticação, responsável pelo controle de sessões, validação de credenciais e gerenciamento seguro de acesso. Os dados de autenticação e informações persistentes são armazenados no Firebase, que atua como serviço de backend para persistência de dados.

O módulo Admin é responsável pela gestão administrativa da aplicação, incluindo o cadastro e controle de Tipos de Carro, que influenciam diretamente na definição de tarifas e regras de atendimento. Esses dados são utilizados pelo sistema no momento da solicitação de corridas.

A funcionalidade de solicitação e execução de corridas é composta por diversos componentes integrados. O módulo Localização é utilizado para determinar a posição do usuário e do motorista, sendo fundamental para a seleção de corridas e cálculo de trajetos. A partir dessas informações, o componente Selecciona Tarifa define o valor da corrida com base nos parâmetros configurados, enquanto o componente Seleccionar Usuário estabelece a associação entre motorista e passageiro.

Durante o fluxo da corrida, o componente Carro Chega representa o evento de chegada do motorista ao local do usuário, integrando-se ao fluxo operacional do sistema. Após a execução da corrida, o componente Finalizar Corrida é responsável por encerrar o atendimento, registrar as informações finais e encaminhar os dados para a etapa de pagamento.

O módulo Sessão Checkout gerencia o processo de pagamento, realizando a comunicação com o componente Conta, responsável pelo controle financeiro. Esse fluxo garante que a corrida seja devidamente finalizada e registrada, assegurando a consistência das informações financeiras do sistema.

De forma geral, o diagrama demonstra uma arquitetura modular e bem definida, com separação clara de responsabilidades entre os componentes. Essa organização favorece a manutenção, a escalabilidade e a evolução futura da aplicação MobU, permitindo a inclusão de novos serviços e funcionalidades de maneira estruturada e controlada.

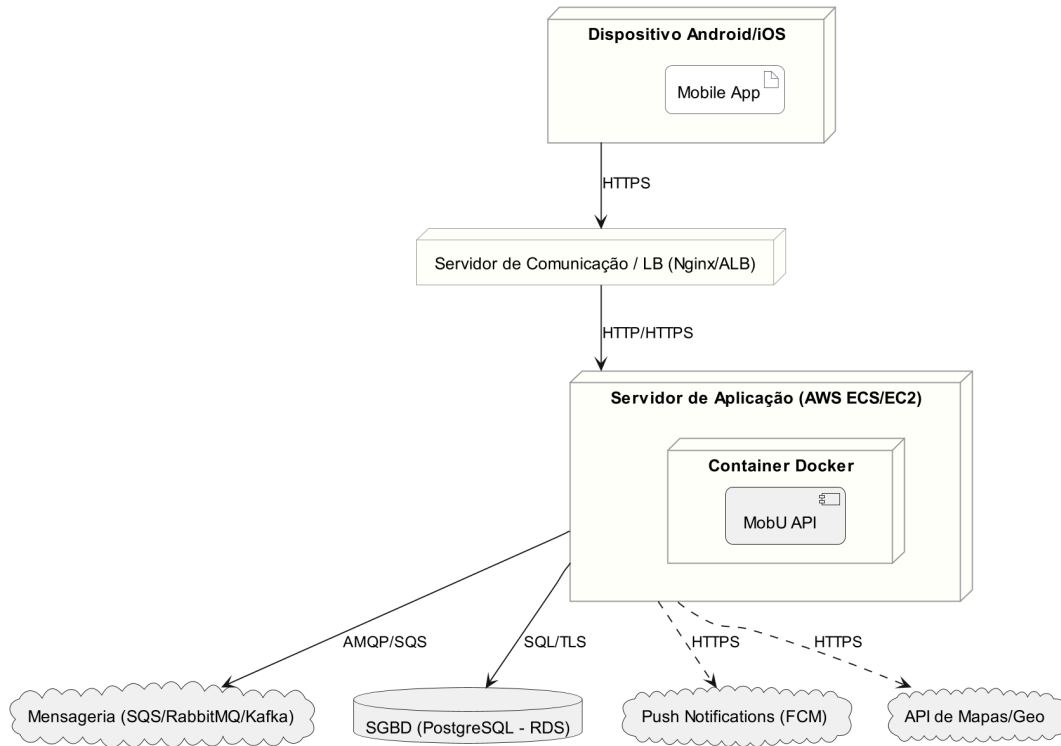


Figura 24 – Diagrama de Implantação da Aplicação

4. Projeto de Interface com Usuário

Esta seção tem como objetivo apresentar e descrever as interfaces de interação com o usuário que compõem o sistema MobU. Para isso, foi elaborado um protótipo de baixa fidelidade, desenvolvido com o intuito de representar a estrutura visual e o fluxo principal de navegação da aplicação. As telas foram construídas de forma simplificada, com foco na organização dos elementos e na disposição das funcionalidades, possibilitando a validação inicial da proposta de interface antes da etapa de design detalhado. Dessa forma, as interfaces aqui apresentadas estão relacionadas aos casos de uso descritos na Seção 2.3.1, servindo como base para o mapeamento das funcionalidades essenciais que atenderão aos requisitos funcionais e não funcionais do sistema.

O objetivo deste protótipo é demonstrar a interação prevista entre os usuários e o sistema, permitindo visualizar o comportamento esperado das principais telas e apoiar o planejamento da fase de implementação.

4.1 Esboço das Interfaces Comuns a Todos os Atores

Wireframe/mockup/storyboard das interfaces que são comuns a todos os atores do sistema.

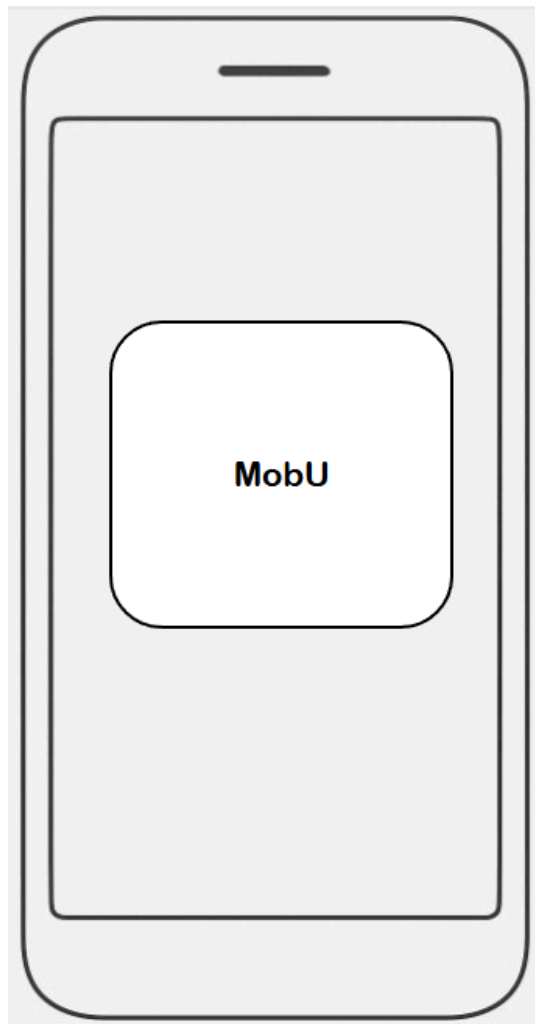


Figura 25. Página Splash Screen.



Figura 26. Página de login/cadastro.

A Figura 25 representa a tela de Splash Screen do aplicativo do passageiro. Esta página é exibida brevemente ao abrir o app e enquanto o sistema carrega dados de sessão ou verifica se o usuário já está autenticado. Caso a autenticação seja válida, a aplicação redireciona automaticamente o usuário para a página principal de solicitação de corrida (Figura 37). A Figura 26 mostra a tela de login/cadastro do usuário.

4.2 Esboço das Interfaces Usadas pelo Administrador

Wireframe/mockup/storyboard das interfaces exclusivas do Administrador.

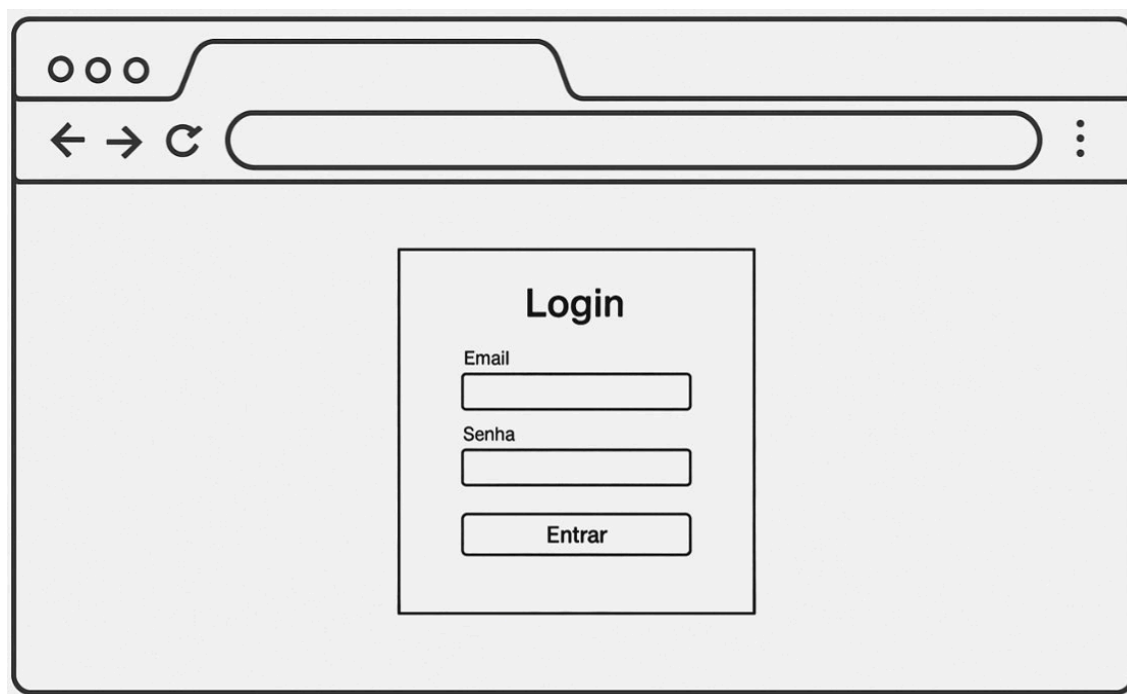


Figura 27. Página de login do administrador.

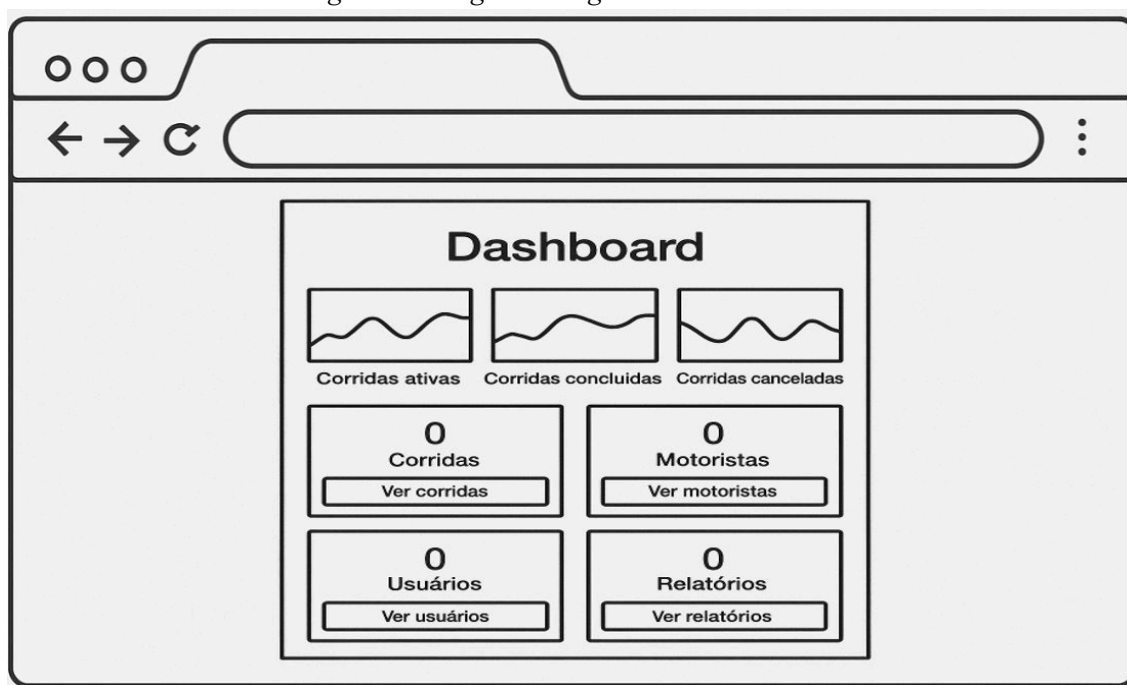


Figura 28. Página do dashboard principal.

A Figura 27 representa a tela de login do administrador, que dá acesso à tela de dashboard principal (Figura 28). O dashboard contém gráficos de corridas ativas, concluídas e canceladas, além de atalhos para funcionalidades administrativas.

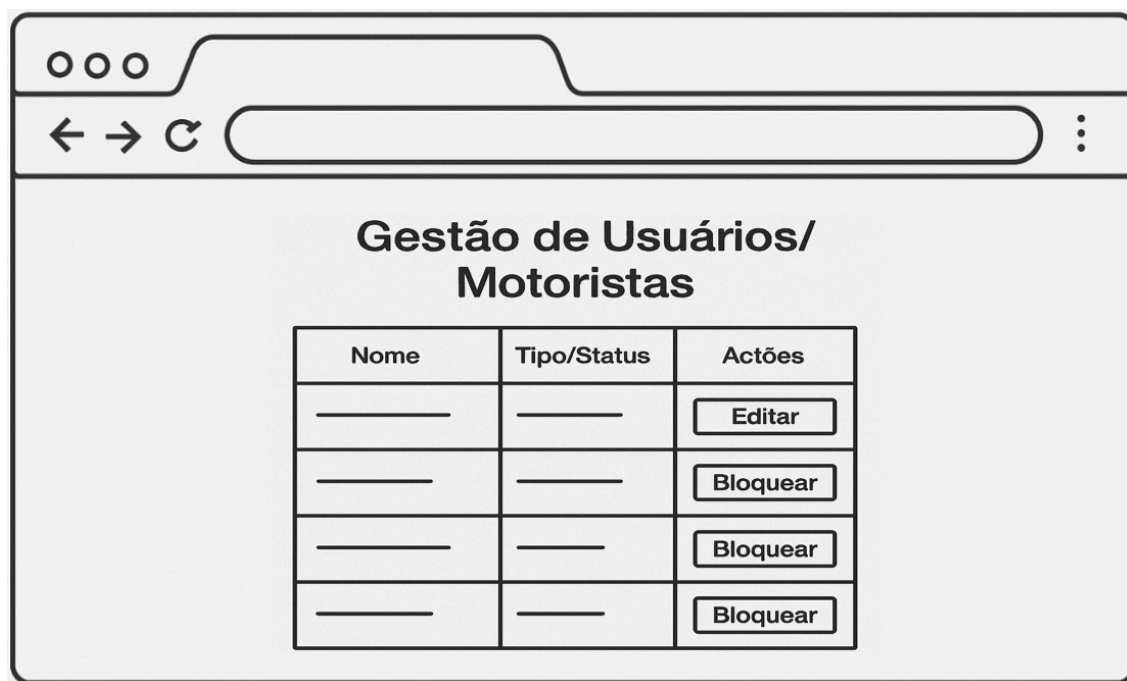


Figura 29. Página da gestão de usuários.

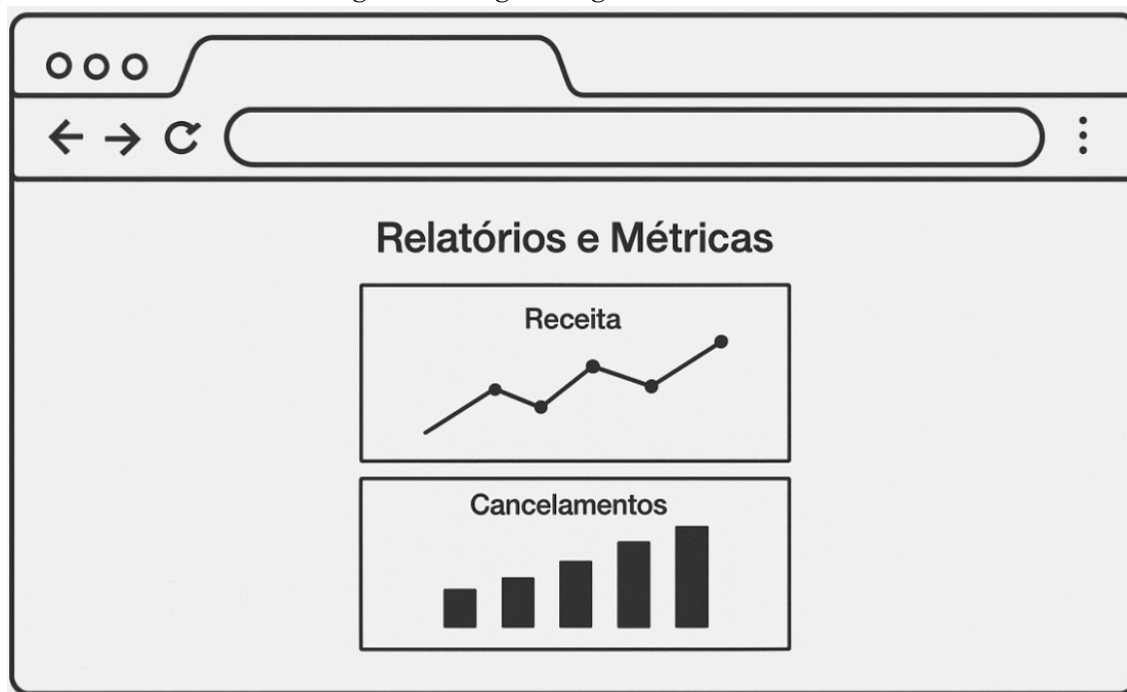


Figura 30. Página de relatórios e métricas.

A Figura 29 apresenta a tela de gestão de usuários e motoristas, com tabela listando os registros, status e botões para editar ou bloquear perfis. A Figura 30 mostra a tela de relatórios e métricas, que contém gráficos sobre receitas, taxas de cancelamento e desempenho do sistema.

The mockup shows a web browser window with a single tab. The address bar contains navigation icons (back, forward, refresh) and a search icon. The main content area is titled "Configuração de Taxas / Regiões". Below the title is a form with two sections: "Porcentagem" with a single text input field, and "Regiões" with four stacked text input fields.

Figura 31. Página de configuração de taxas e regiões.

The mockup shows a web browser window with a single tab. The address bar contains navigation icons (back, forward, refresh) and a search icon. The main content area is titled "Suporte / Disputas". Below the title is a form with four rows, each containing a text input field and a button. The buttons are labeled "Responder" for the first two rows and "Encerrar" for the last two rows.

Figura 32. Página de suporte e disputas.

A Figura 31 apresenta a tela de configuração de taxas e regiões, na qual o administrador define percentuais de comissão e gerencia as áreas atendidas pela plataforma. Por fim, a Figura 32 exibe a tela de suporte e disputas, que lista chamados abertos por usuários ou motoristas e permite ao administrador registrar respostas ou encerrar cada caso.

4.3 Esboço das Interfaces Usadas pelo Motorista

Wireframe/mockup/storyboard das interfaces exclusivas do Motorista.



Figura 33. Página de status do motorista.



Figura 34. Página de nova corrida.

Após acessar a conta, o motorista é direcionado para a tela de status online/offline (Figura 33), que possui um botão central destacando o estado atual (verde para disponível, vermelho para indisponível). Quando o motorista está online e recebe uma nova corrida, é exibida a tela de solicitação (Figura 34), que contém os dados do passageiro, origem, destino e valor estimado, além dos botões “Aceitar” e “Recusar”.



Figura 35. Página de navegação do motorista.



Figura 36. Página relatório de ganhos.

Uma vez aceita a corrida, o app abre a tela de navegação (Figura 35), que mostra no mapa a rota até o local de embarque e, posteriormente, até o destino do passageiro. Nesta tela, o motorista também tem acesso aos botões “Cheguei”, “Iniciar viagem” e “Finalizar viagem” (Figura 35), cada um alterando o estado da corrida. Concluída a viagem, o motorista é redirecionado à tela de relatórios de ganhos (Figura 36), que apresenta dados de corridas finalizadas, ganhos do dia e gráficos semanais.

4.4 Esboço das Interfaces Usadas pelo Passageiro

Wireframe/mockup/storyboard das interfaces exclusivas do Passageiro.

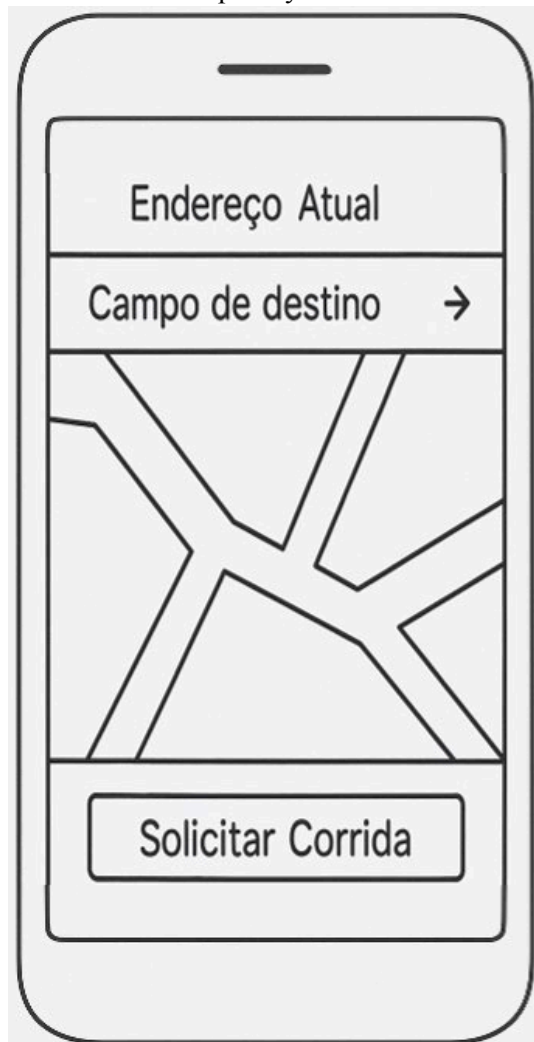


Figura 37. Página inicial do passageiro.



Figura 38. Página estimativa de corrida.

A Figura 37 representa a tela principal de solicitação de corrida. Nela o passageiro vê um mapa com a posição atual, campo para inserir o destino e botão para solicitar corrida. Após preencher o destino, o usuário é redirecionado para a tela de estimativa de tarifa (Figura 38), que mostra o valor previsto da viagem e tempo de chegada do motorista. A partir desta tela, o passageiro pode confirmar ou cancelar a solicitação.



Figura 39. Página de acompanhamento.

A Figura 39 apresenta a tela de acompanhamento da corrida, que exibe no mapa a posição do motorista em tempo real, dados do motorista e do veículo, além de um botão para cancelamento da corrida antes do início.

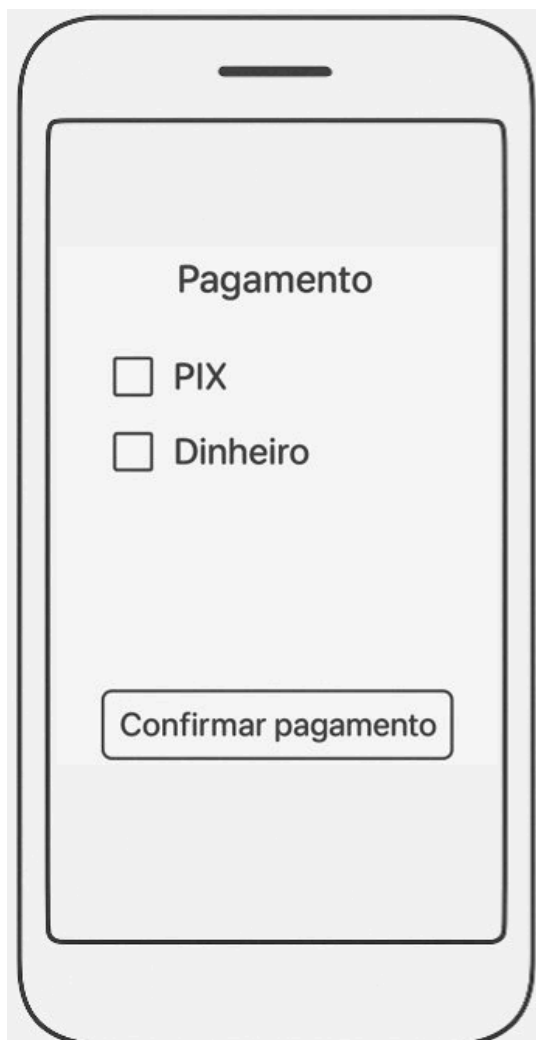


Figura 40. Página forma de pagamento.



Figura 41. Página perfil do usuário.

Após a finalização da viagem, o passageiro é redirecionado à tela de pagamento (Figura 40), onde escolhe entre PIX ou dinheiro. A Figura 41 mostra a tela de perfil do usuário, na qual é possível editar informações pessoais, acessar o histórico de corridas e encerrar a sessão. O histórico apresenta uma lista de viagens anteriores com informações resumidas como data, valor e status.

5. Glossário e Modelos de Dados

Esta seção tem como objetivo definir o glossário de termos técnicos e de domínio utilizados no sistema MobU, bem como descrever em detalhes o modelo de dados implementado. A Tabela 8 apresenta o glossário com os principais conceitos da aplicação. As Tabelas 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 e 21 descrevem, respectivamente, as entidades User, DriverProfile, Ride, PricingConfig, OperationRegion, Review, ChatMessage, SupportTicket, PromoCode,

DriverPaymentRequest, DriverSettlement, RideAuditLog e SystemConfig, com seus campos e tipos conforme definido no schema Prisma do sistema. Por fim, a Tabela 22 consolida as enumerações utilizadas no banco de dados.

Termo	Definição
MobU	Aplicação de transporte urbano por aplicativo desenvolvida como objeto deste trabalho, composta por app Android para passageiros e motoristas, API NestJS e painel administrativo Next.js.
Passageiro	Usuário com perfil <code>PASSENGER</code> que solicita corridas pelo aplicativo móvel.
Motorista	Usuário com perfil <code>DRIVER</code> cadastrado, aprovado e apto a aceitar e executar corridas.
Administrador	Usuário com perfil <code>ADMIN</code> que acessa o painel web para gerenciar usuários, corridas, precificações e pagamentos.
Corrida	Entidade central do sistema que representa a solicitação de transporte desde a requisição do passageiro até a conclusão ou cancelamento.
Status da corrida	Estado atual de uma corrida, podendo ser: <code>PENDING_DRIVER</code> , <code>ACCEPTED</code> , <code>DRIVER_ARRIVING</code> , <code>DRIVER_ARRIVED</code> , <code>IN_PROGRESS</code> , <code>FINISHED</code> ou <code>CANCELED</code> .
Perfil do motorista	Conjunto de dados complementares ao usuário motorista, incluindo CNH, veículo, localização em tempo real e chave Pix.
Aprovação de motorista	Processo pelo qual o administrador analisa e aprova ou rejeita o cadastro de um motorista. Estados: <code>PENDING</code> , <code>APPROVED</code> , <code>REJECTED</code> .

Configuração o de precificação	Conjunto de parâmetros tarifários (tarifa base, valor por km, valor por minuto, multiplicador de demanda) aplicados ao cálculo de corridas.
Região de operação	Área geográfica delimitada (centro + raio em metros) na qual o sistema pode operar, associada a uma configuração de precificação específica.
Avaliação	Nota (1–5) e comentário opcional atribuídos pelo passageiro ao motorista ao final de uma corrida.
Chat de corrida	Canal de mensagens em tempo real entre passageiro e motorista durante uma corrida ativa, persistido no banco de dados.
Ticket de suporte	Solicitação de ajuda aberta por um usuário relacionada a uma corrida ou problema no aplicativo, gerenciada pelo administrador.
Código promocional	Código alfanumérico que concede desconto percentual ou fixo em corridas, com controle de usos máximos e validade.
Rejeição de corrida	Registro de que um motorista recusou uma corrida pendente, impedindo que a mesma corrida seja ofertada novamente ao mesmo motorista.
Solicitação de pagamento	Pedido formal de repasse financeiro feito pelo motorista ao administrador, referente ao saldo acumulado de corridas.
Liquidação	Confirmação pelo administrador de que o repasse financeiro ao motorista foi realizado, vinculada a uma solicitação de pagamento.
Log de auditoria	Registro imutável de cada ação significativa sobre uma corrida (aceitação, início, fim, cancelamento, etc.), com identificação do ator e metadados.

FCM Token	Identificador único gerado pelo Firebase Cloud Messaging que permite o envio de notificações push ao dispositivo do usuário.
Pix	Sistema de pagamento instantâneo do Banco Central do Brasil, suportado como método de pagamento de corridas e repasse a motoristas.
Tarifa base	Valor fixo cobrado no início de toda corrida, independentemente da distância ou duração.
Taxa de plataforma	Percentual retido pela plataforma MobU sobre o valor total de cada corrida antes do repasse ao motorista.
Multiplicador de demanda	Fator aplicado à tarifa em períodos de alta demanda (<i>surge pricing</i>). Valor 1,0 indica sem acréscimo.
NestJS	Framework Node.js baseado em TypeScript utilizado para construção da API REST do sistema MobU.
Prisma ORM	Object-Relational Mapper utilizado para modelagem, migração e acesso ao banco de dados PostgreSQL a partir da API.
PostgreSQL	Sistema gerenciador de banco de dados relacional utilizado como camada de persistência do sistema MobU.
FCM (Firebase Cloud Messaging)	Serviço do Google utilizado exclusivamente para envio de notificações push aos dispositivos Android dos usuários.
JWT (JSON Web Token)	Padrão de token utilizado para autenticação e autorização nas requisições à API após o login.

WebSocket	Protocolo de comunicação bidirecional utilizado para atualizações em tempo real de localização do motorista e eventos de corrida.
API REST	Interface de programação baseada no protocolo HTTP que expõe os recursos do sistema MobU para os clientes (app Android e painel web).
UUID / CUID	Formatos de identificadores únicos utilizados como chave primária das entidades do banco de dados.
CNH	Carteira Nacional de Habilitação, documento obrigatório para cadastro de motoristas na plataforma.
Jetpack Compose	Toolkit declarativo do Android utilizado para construção das interfaces do aplicativo móvel MobU.

Tabela 8 – Glossário de termos utilizados no sistema MobU

Campo	Tipo	Descrição
id	String (UUID)	Identificador único do usuário.
phone	String	Número de telefone único, utilizado como login.
name	String	Nome completo do usuário.
email	String	Endereço de e-mail único do usuário.

passwordHash	String	Hash bcrypt da senha definida no cadastro.
role	Enum UserRole	Perfil do usuário: PASSENGER, DRIVER ou ADMIN.
profilePhotoUrl	String	URL da foto de perfil.
fcmToken	String	Token do Firebase Cloud Messaging para notificações push.
blocked	Boolean	Indica se o usuário está bloqueado (padrão: false).
blockedAt	DateTime	Data e hora em que o usuário foi bloqueado.
phoneVerifiedAt	DateTime	Data de verificação do telefone.
emailVerifiedAt	DateTime	Data de verificação do e-mail.
createdAt	DateTime	Data e hora de criação do registro.

Tabela 9 – Entidade User

Campo	Tipo	Descrição
id	String (CUID)	Identificador único do perfil.

userId	String	Referência ao usuário proprietário do perfil (único).
approvalStatus	Enum DriverApprovalStatus	Estado de aprovação: PENDING, APPROVED ou REJECTED.
approvedAt	DateTime	Data de aprovação pelo administrador.
rejectionReason	String	Motivo da rejeição do cadastro.
online	Boolean	Indica se o motorista está online no momento.
available	Boolean	Indica se o motorista está disponível para aceitar corridas.
currentLat	Float	Latitude atual do motorista (atualizada em tempo real).
currentLng	Float	Longitude atual do motorista.
cpf	String	CPF do motorista.
cnhNumber	String	Número da CNH.
cnhCategory	String	Categoria da CNH (ex.: B).
cnhExpiresAt	DateTime	Data de validade da CNH.

<code>cnhImageUrl</code>	String	URL da imagem da CNH enviada pelo motorista.
<code>vehicleModel</code>	String	Modelo do veículo (ex.: Onix).
<code>vehiclePlate</code>	String	Placa do veículo.
<code>vehicleColor</code>	String	Cor do veículo.
<code>vehicleYear</code>	Int	Ano de fabricação do veículo.
<code>vehicleCapacity</code>	Int	Capacidade de passageiros do veículo.
<code>hasEar</code>	Boolean	Indica se o motorista possui credencial EAR (especial).
<code>pixKey</code>	String	Chave Pix do motorista para recebimento de repasses.
<code>pixQrPayload</code>	String	Payload do QR Code Pix gerado.
<code>pixQrCodeUrl</code>	String	URL da imagem do QR Code Pix.
<code>profilePhotoUrl</code>	String	URL da foto de perfil do motorista.
<code>createdAt</code>	DateTime	Data de criação do perfil.

updatedAt	DateTime	Data da última atualização.
-----------	----------	-----------------------------

Tabela 10 – Entidade DriverProfile

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da corrida.
passengerId	String	Referência ao usuário passageiro.
driverId	String	Referência ao usuário motorista (preenchido após aceitação).
status	Enum RideStatus	Estado atual da corrida (ver enumeração RideStatus).
originLat originLng	Float	Coordenadas geográficas do ponto de origem.
destLat / destLng	Float	Coordenadas geográficas do destino.
originAddress	String	Endereço textual do ponto de origem.
destinationAddress	String	Endereço textual do destino.

distanceMeters	Int	Distância estimada em metros.
durationSeconds	Int	Duração estimada em segundos.
estimatedFareCents	Int	Valor estimado da corrida em centavos.
driverReceivableCents	Int	Valor a receber pelo motorista após desconto da taxa de plataforma.
platformFeeCents	Int	Valor retido pela plataforma em centavos.
pricingConfigId	String	Configuração de precificação aplicada à corrida.
paymentMethod	Enum PaymentMethod	Método de pagamento: CASH ou PIX.
paymentStatus	Enum PaymentStatus	Status do pagamento: PENDING, RECEIVED, NOT_RECEIVED ou CANCELED.
paymentPixPayload	String	Payload do QR Code Pix para pagamento da corrida.
paymentTxId	String	ID da transação Pix.
shareToken	String	Token único para compartilhamento de localização da corrida.

acceptedAt	DateTime	Data/hora em que o motorista aceitou a corrida.
driverArrivingAt	DateTime	Data/hora em que o motorista iniciou o deslocamento ao ponto de origem.
driverArrivedAt	DateTime	Data/hora em que o motorista chegou ao ponto de origem.
startedAt	DateTime	Data/hora de início da corrida.
finishedAt	DateTime	Data/hora de conclusão da corrida.
canceledAt	DateTime	Data/hora de cancelamento.
createdAt	DateTime	Data/hora de criação da solicitação.
updatedAt	DateTime	Data/hora da última atualização.

Tabela 11 – Entidade Ride

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da configuração.
name	String	Nome descritivo da configuração (ex.: "Tarifa padrão").

isActive	Boolean	Indica se a configuração está em uso.
regionId	String	Região de operação associada.
baseFareCents	Int	Tarifa base em centavos (padrão: 500).
perKmCents	Int	Valor por quilômetro em centavos (padrão: 200).
perMinuteCents	Int	Valor por minuto em centavos (padrão: 50).
minimumFareCents	Int	Tarifa mínima em centavos (padrão: 800).
bookingFeeCents	Int	Taxa de reserva em centavos (padrão: 0).
surgeMultiplier	Float	Multiplicador de demanda (padrão: 1,0 = sem acréscimo).
platformFeePercent	Float	Percentual de taxa da plataforma (padrão: 20%).
currency	String	Moeda utilizada (padrão: "BRL").
createdAt / updatedAt	DateTime	Datas de criação e atualização.

Tabela 12 – Entidade PricingConfig

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da região.
name	String	Nome da região (ex.: "Centro de Belo Horizonte").
city	String	Cidade da região.
active	Boolean	Indica se a região está ativa.
centerLat / centerLng	Float	Coordenadas do centro geográfico da região.
radiusMeters	Int	Raio de cobertura em metros.
createdAt / updatedAt	DateTime	Datas de criação e atualização.

Tabela 13 – Entidade OperationRegion

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da avaliação.
rideId	String	Corrida avaliada (único — uma corrida possui no máximo uma avaliação).

passengerId	String	Passageiro que realizou a avaliação.
driverId	String	Motorista avaliado.
rating	Int	Nota de 1 a 5.
comment	String	Comentário textual da avaliação.
createdAt	DateTime	Data de criação.

Tabela 14 – Entidade Review

Campo	Tipo	Descrição
id	String (CUID)	Identificador único da mensagem.
rideId	String	Corrida à qual a mensagem pertence.
senderId	String	ID do usuário remetente.
senderRole	String	Papel do remetente: "PASSENGER" ou "DRIVER".
content	String	Conteúdo textual da mensagem.

sentAt	DateTime	Data/hora de envio.
readAt	DateTime	Data/hora de leitura pelo destinatário.

Tabela 15 – Entidade ChatMessage

Campo	Tipo	Descrição
id	String (UUID)	Identificador único do ticket.
creatorId	String	Usuário que abriu o ticket.
rideId	String	Corrida relacionada ao ticket (se aplicável).
type	Enum SupportTicketType	Categoria: PAYMENT, RIDE_CANCELLATION, SAFETY, APP_ISSUE ou OTHER.
status	Enum SupportTicketStatus	Estado: OPEN, IN_REVIEW, RESOLVED ou CLOSED.
subject	String	Assunto do ticket.
description	String	Descrição detalhada do problema.
resolution	String	Descrição da resolução pelo administrador.

<code>closedAt</code>	<code>DateTime</code>	Data/hora de encerramento do ticket.
<code>createdAt</code> / <code>updatedAt</code>	<code>DateTime</code>	Datas de criação e atualização.

Tabela 16 – Entidade *SupportTicket*

Campo	Tipo	Descrição
<code>id</code>	String (CUID)	Identificador único.
<code>code</code>	String	Código alfanumérico único (ex.: "PROMO10").
<code>discountPercent</code>	Int	Percentual de desconto (0–100).
<code>discountCents</code>	Int	Valor fixo de desconto em centavos.
<code>maxUses</code>	Int	Número máximo de utilizações (padrão: 1).
<code>usedCount</code>	Int	Número de vezes que o código foi utilizado.
<code>active</code>	Boolean	Indica se o código está ativo.
<code>expiresAt</code>	<code>DateTime</code>	Data de expiração do código.

createdAt	DateTime	Data de criação.
-----------	----------	------------------

Tabela 17 – Entidade PromoCode

Campo	Tipo	Descrição
id	String (UUID)	Identificador único.
driverId	String	Motorista solicitante.
amountCents	Int	Valor solicitado em centavos.
status	Enum PaymentRequestStatus	Estado: PENDING, CONFIRMED ou REJECTED.
notes	String	Observações do motorista.
receiptUrl	String	URL do comprovante de pagamento.
rejectionReason	String	Motivo de rejeição pelo administrador.
requestedAt	DateTime	Data/hora da solicitação.
reviewedAt	DateTime	Data/hora da revisão pelo administrador.

reviewedBy	String	ID do administrador que revisou.
------------	--------	----------------------------------

Tabela 18 – Entidade DriverPaymentRequest

Campo	Tipo	Descrição
id	String (UUID)	Identificador único.
driverId	String	Motorista beneficiário do repasse.
amountCents	Int	Valor repassado em centavos.
method	String	Método de repasse (padrão: "PIX").
notes	String	Observações do administrador.
paymentRequestId	String	Solicitação de pagamento vinculada (único).
settledBy	String	ID do administrador que confirmou o repasse.
settledAt	DateTime	Data/hora do repasse.

Tabela 19 – Entidade DriverSettlement

Campo	Tipo	Descrição
id	String (CUID)	Identificador único do log.
rideId	String	Corrida à qual o log se refere.
actorUserId	String	ID do usuário que realizou a ação (nulo para ações automáticas do sistema).
action	String	Descrição da ação registrada (ex.: "RIDE_ACCEPTED", "RIDE_STARTED").
metadata	JSON	Dados adicionais da ação em formato JSON.
createdAt	DateTime	Data/hora do registro.

Tabela 20 – Entidade RideAuditLog

Campo	Tipo	Descrição
key	String	Chave de identificação da configuração (chave primária).
value	String	Valor da configuração.

updatedAt	DateTime	Data da última atualização.
-----------	----------	-----------------------------

Tabela 21 – Entidade SystemConfig

Enumeração	Valores	Descrição
UserRole	PASSENGER, DRIVER, ADMIN	Perfil de acesso do usuário no sistema.
RideStatus	PENDING_DRIVER, ACCEPTED, DRIVER_ARRIVING, DRIVER_ARRIVED, IN_PROGRESS, FINISHED, CANCELED	Estados possíveis de uma corrida.
PaymentMethod	CASH, PIX	Método de pagamento da corrida.
PaymentStatus	PENDING, RECEIVED, NOT_RECEIVED, CANCELED	Status do pagamento da corrida.
DriverApprovalStatus	PENDING, APPROVED, REJECTED	Estado de aprovação do cadastro de motorista.
PaymentRequestStatus	PENDING, CONFIRMED, REJECTED	Estado de uma solicitação de pagamento do motorista.
SupportTicketType	PAYMENT, RIDE_CANCELLATION, SAFETY, APP_ISSUE, OTHER	Categoria do ticket de suporte.

SupportTicketStatus	OPEN, IN_REVIEW, RESOLVED, CLOSED	Estado do ticket de suporte.
VerificationChannel	SMS, EMAIL	Canal utilizado para desafios de verificação de identidade.

Tabela 22 – Enumerações do banco de dados

Da mesma forma, a Figura 42 tem como objetivo representar a camada de banco de dados da aplicação por meio de um Diagrama Entidade-Relacionamento (DER). Nesse diagrama, é possível identificar todas as entidades implementadas, tais como Usuário, Motorista, Veículo, Corrida, Localização, Pagamento e Avaliação, bem como a maneira pela qual elas se relacionam.

O modelo de dados do MobU foi estruturado para garantir consistência, integridade e rastreabilidade das informações. Cada corrida é associada a um passageiro e motorista, contendo dados de origem, destino, valor e status operacional. Os pagamentos armazenam o método utilizado (PIX ou dinheiro), o status da transação e os dados de confirmação. Já as avaliações permitem o registro da nota e do comentário referente à experiência de corrida.

Por fim, o veículo é vinculado diretamente ao motorista e contém as informações de identificação e uso dentro do sistema.

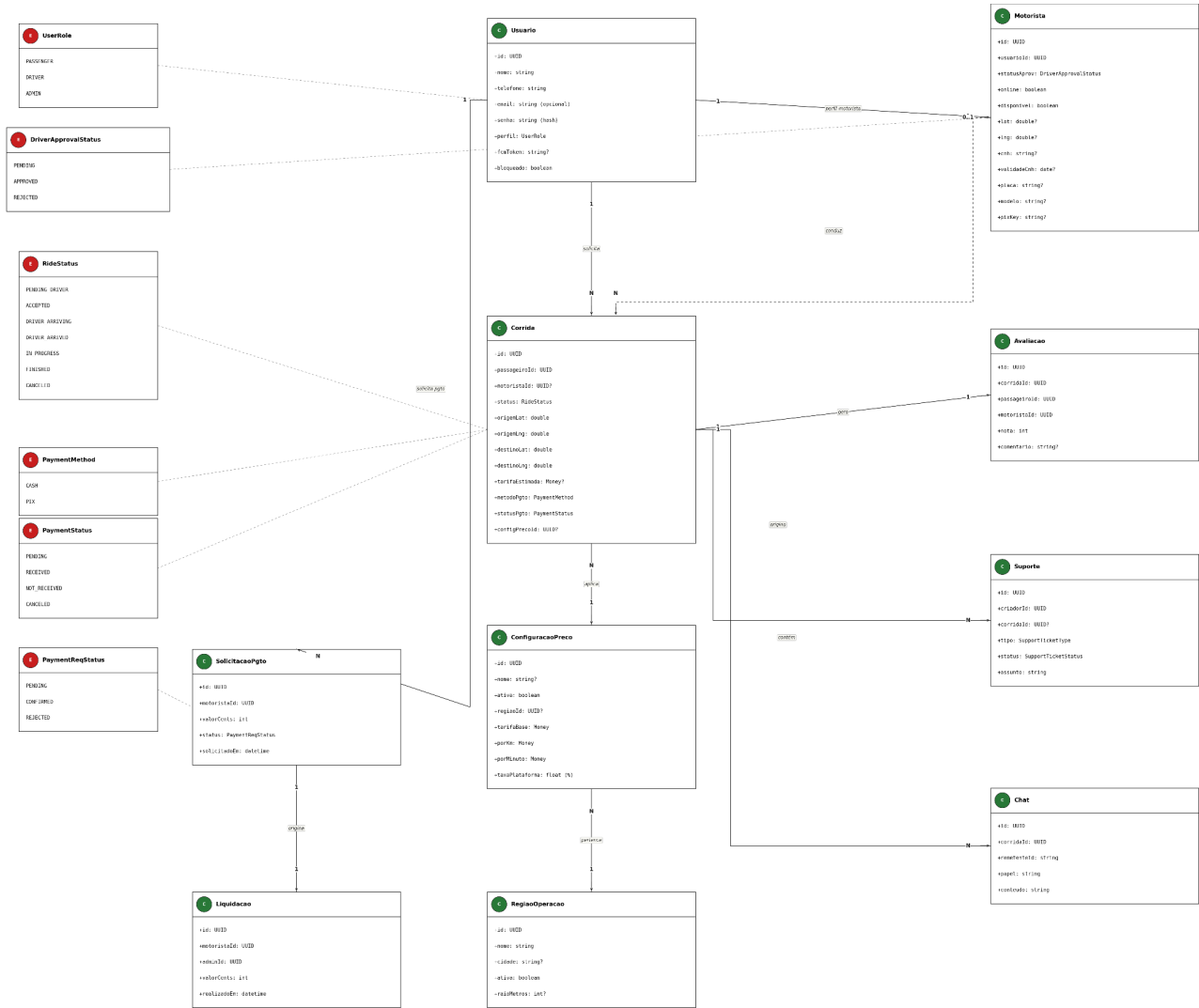


Figura 42. Diagrama de Entidade e Relacionamento.

6. Casos de Teste

Esta seção tem como objetivo apresentar os testes de aceitação planejados para o sistema MobU, garantindo que as funcionalidades previstas no Documento de Visão sejam devidamente verificadas. Os testes aqui definidos serão implementados futuramente e têm como propósito assegurar que o sistema atenda às necessidades essenciais de seus usuários — passageiros e motoristas — permitindo o uso correto e seguro dos principais fluxos da aplicação.

Cada teste de aceitação é descrito por meio de uma tabela contendo quatro campos principais: identificador, pré-condição, ações e resultados esperados. O identificador permite referenciar o teste de maneira única. A pré-condição define o estado necessário para que o teste possa ser iniciado. As

ações descrevem passo a passo as interações do usuário na aplicação. Por fim, os resultados descrevem o comportamento esperado do sistema após a execução do teste.

Esses testes serão utilizados futuramente para validar a experiência do usuário, confirmar o correto funcionamento dos fluxos críticos e demonstrar que o sistema opera conforme os requisitos estabelecidos para a primeira entrega funcional do MobU. Todos os testes apresentados foram definidos com base nas necessidades identificadas durante o processo de levantamento de requisitos e serão executados conforme o progresso do desenvolvimento.

6.1 Testes de Aceitação — Necessidade 1: Passageiro solicitar corrida

A primeira necessidade prevê que o passageiro seja capaz de solicitar uma corrida informando origem e destino. Para essa necessidade, foram definidos três testes de aceitação, relacionados à criação da conta, à solicitação da corrida e à visualização do motorista no mapa.

Caso de Teste de Aceitação 1: Passageiro deve conseguir criar conta no MobU	
Identificador	TA1
Pré-condição	O passageiro não deve possuir cadastro prévio no MobU.
Ações	1. Inserir número de telefone. 2. Inserir código de validação enviado por SMS. 3. Informar nome, e-mail e aceitar os termos de uso.
Resultados	Resultado esperado: a conta deve ser criada e o usuário deve ser redirecionado ao dashboard inicial. Resultado obtido: aprovado. O cadastro/autenticação de passageiro foi validado pela API e o login foi confirmado no aplicativo Android com usuário QA.

Tabela 16. Caso de teste de aceitação 1

Caso de Teste de Aceitação 2: Passageiro deve conseguir solicitar uma corrida	
Identificador	TA2

Pré-condição	O passageiro deve estar autenticado na aplicação.
Ações	1. Inserir origem e destino. 2. Confirmar os dados da corrida. 3. Selecionar forma de pagamento (dinheiro ou Pix offline). 4. Solicitar corrida.
Resultados	Resultado esperado: o sistema deve registrar a corrida e exibir o status “Aguardando motorista”. Resultado obtido: aprovado. A corrida foi estimada e criada via API dentro da região ativa de operação, com registro persistido e disponibilização para motorista online.

Tabela 17. Caso de teste de aceitação 2

Caso de Teste de Aceitação 3: Passageiro deve conseguir acompanhar o motorista	
Identificador	TA3
Pré-condição	O motorista deve ter aceitado a corrida.
Ações	1. Abrir tela de acompanhamento. 2. Visualizar a posição do motorista no mapa em tempo real.
Resultados	Resultado esperado: o passageiro deve visualizar o motorista se movendo no mapa até o local de embarque. Resultado obtido: aprovado. Após o aceite da corrida, a atualização de localização do motorista foi persistida e consultada pelo passageiro durante o acompanhamento.

Tabela 18. Caso de teste de aceitação 3

6.1.2 Necessidade 2 — Motorista aceitar e realizar corrida

A segunda necessidade garante que o motorista seja capaz de receber, aceitar e concluir corridas, validando o funcionamento da API, os eventos e a comunicação entre os módulos.

Caso de Teste de Aceitação 4: Motorista aceitar corrida	
Identificador	TA4
Pré-condição	O motorista deve estar autenticado e marcado como “online”.
Ações	1. Receber notificação de corrida. 2. Visualizar origem e destino. 3. Aceitar corrida.
Resultados	Resultado esperado: a corrida passa para o estado “Aceita” e o motorista passa para o modo de navegação. Resultado obtido: aprovado. Motorista online visualizou a corrida pendente, aceitou a solicitação e a API vinculou a corrida ao motorista com mudança de estado.

Tabela 19. Caso de teste de aceitação 4

Caso de Teste de Aceitação 5: Motorista finalizar corrida	
Identificador	TA5
Pré-condição	A corrida deve estar em andamento.
Ações	1. Marcar chegada ao destino. 2. Finalizar corrida. 3. Confirmar valor final com o passageiro.
Resultados	Resultado esperado: a corrida deve ser finalizada e registrada no histórico do motorista e do passageiro. Resultado obtido: aprovado. O fluxo de status chegando, chegou, em andamento e finalizada foi concluído, e a corrida apareceu nos históricos de motorista e passageiro.

Tabela 20. Caso de teste de aceitação 5

Caso de Teste de Aceitação 6: O Motorista deve ser capaz de gerenciar seu status (online/offline)	
Identificador	TA6

Pré-condição	O motorista deve estar autenticado na aplicação.
Ações	1. Acessar o painel inicial. 2. Selecionar o botão “Ficar Online”.
Resultados	Resultado esperado: o motorista passa a receber solicitações de corrida e seu status é atualizado no sistema. Resultado obtido: aprovado. A alteração de status online/off-line foi persistida pela API e considerada na distribuição/listagem de corridas disponíveis.

Tabela 21. Caso de teste de aceitação 6

6.1.3 Necessidade 3 — Pagamento via PIX Off-line

Esta necessidade refere-se ao processo de geração de QR Code estático utilizando chave Pix previamente cadastrada no banco de dados, sem uso de API de instituição financeira.

Caso de Teste de Aceitação 7: Gerar QR Code Pix estático	
Identificador	TA7
Pré-condição	O motorista deve possuir chave Pix cadastrada no sistema
Ações	1. Finalizar corrida. 2. Selecionar pagamento via Pix. 3. Gerar QR Code com valor.
Resultados	Resultado esperado: o QR Code deve ser exibido corretamente e pode ser pago pelo aplicativo bancário do passageiro. Resultado obtido: aprovado. A API gerou payload Pix/QR Code estático com valor da corrida e txId associado ao pagamento.

Tabela 22. Caso de teste de aceitação 7

Caso de Teste de Aceitação 8: Registrar pagamento não confirmado	
Identificador	TA8

Pré-condição	A corrida deve estar finalizada e aguardando validação de pagamento.
Ações	1. Selecionar corrida finalizada. 2. Informar “Pagamento não recebido”.
Resultados	Resultado esperado: o sistema marca o pagamento como “Pendente”, registrando data e motorista responsável. Resultado obtido: aprovado. Ao marcar pagamento não recebido, a API registrou a pendência/não confirmação mantendo rastreabilidade para validação posterior.

Tabela 23. Caso de teste de aceitação 8

Caso de Teste de Aceitação 9: Visualizar comprovante do PIX	
Identificador	TA9
Pré-condição	O pagamento da corrida deve estar marcado como “Pago”.
Ações	1. Abrir detalhes da corrida. 2. Acessar a área de pagamentos.
Resultados	Resultado esperado: o comprovante (txid + dados da cobrança) é exibido ao motorista e ao passageiro. Resultado obtido: aprovado. Após confirmação do pagamento, os detalhes da corrida exibiram txId e payload Pix para consulta no histórico.

Tabela 24. Caso de teste de aceitação 9

6.1.4 Necessidade 4 — Administração de Usuários e Monitoramento do Sistema

Esta necessidade refere-se às funcionalidades destinadas ao administrador da plataforma MobU, permitindo a gestão centralizada de usuários e o acompanhamento operacional do sistema. O administrador deve ser capaz de visualizar, monitorar e realizar ações de controle sobre contas de passageiros e motoristas, bem como acessar relatórios consolidados de uso da aplicação.

Caso de Teste de Aceitação 10: O Administrador deve gerenciar motoristas e passageiros

Identificador	TA10
Pré-condição	O administrador deve estar autenticado no painel web.
Ações	1. Abrir a aba “Usuários”.
Resultados	Resultado esperado: o sistema exibe lista com passageiros e motoristas cadastrados. Resultado obtido: aprovado. Painel/admin/API retornaram listas de passageiros e motoristas cadastrados, incluindo os usuários QA criados na execução.

Tabela 25. Caso de teste de aceitação 10

Caso de Teste de Aceitação 11: Bloquear usuário	
Identificador	TA11
Pré-condição	Usuário selecionado deve estar ativo no sistema.
Ações	1. Selecionar usuário. 2. Clicar em “Bloquear usuário”
Resultados	Resultado esperado: o usuário fica impedido de acessar o sistema e seu status é marcado como “Bloqueado”. Resultado obtido: aprovado. O bloqueio administrativo alterou o status do usuário para bloqueado e preservou a regra de controle de acesso.

Tabela 26. Caso de teste de aceitação 11

Caso de Teste de Aceitação 12: Visualizar relatórios de corridas	
Identificador	TA12
Pré-condição	O sistema deve conter corridas concluídas.
Ações	1. Acessar a aba “Relatórios”. 2. Selecionar período desejado.

Resultados	Resultado esperado: um relatório com número de corridas, ganhos e estatísticas é exibido ao administrador. Resultado obtido: aprovado. O relatório administrativo retornou indicadores consolidados de corridas, usuários e pagamentos após o fluxo concluído.
-------------------	--

Tabela 27. Caso de teste de aceitação 12

6.2 Testes de Integração

Nesta seção serão descritos os testes de integração previstos para o sistema MobU. Os testes de integração serão responsáveis por verificar se os módulos principais da solução — aplicativo móvel, API MobU e serviços externos (API de Mapas/Geo, serviço de notificações push e futuro gateway de pagamento) — se comunicam corretamente, trocando dados de forma consistente e mantendo o fluxo das corridas do início ao fim.

Cada caso de teste de integração será documentado por meio de uma tabela contendo: identificador, sistemas envolvidos, interface utilizada, pré-condições, entradas e resultados esperados. A partir desses casos, será possível validar cenários críticos como autenticação de usuário, solicitação de corrida com cálculo de rota, envio de notificações para motoristas e registro de pagamentos.

Como estratégia geral, pretende-se adotar uma abordagem top-down. Inicialmente, serão implementadas e testadas as integrações entre a MobU API e os serviços externos, utilizando mocks para simular respostas da API de Mapas/Geo, do serviço de notificações push (FCM). Em um segundo momento, o aplicativo móvel será conectado à API real, permitindo a execução de testes ponta a ponta. A automatização desses testes deverá ser feita por meio de suítes de testes da API e, quando possível, por testes de integração no aplicativo móvel, garantindo que regressões possam ser detectadas de forma rápida ao longo do desenvolvimento.

A Tabela 28 apresenta o caso de teste de integração que valida o processo de autenticação de usuários no MobU. Esse teste verifica a comunicação entre o aplicativo móvel e a API, garantindo que credenciais válidas resultem em geração de token e acesso ao perfil correspondente.

Identificador	TI1 – Autenticação de usuário
Sistemas Envolvidos	Mobile App (Passageiro/Motorista) e MobU API
Interface	Requisição HTTP REST (POST /auth/login) e armazenamento do token JWT no aplicativo.
Pré-condição	Usuário cadastrado no banco de dados e servidor da API em execução.

Entradas	1. Enviar telefone/e-mail e senha válidos a partir da tela de login.
Resultados	Resultado esperado: API deve retornar código 200, token JWT e dados básicos do usuário; o aplicativo deve armazenar o token e redirecionar para a tela inicial do perfil correspondente. Resultado obtido: aprovado. A API retornou 200 com JWT e dados do usuário, e o aplicativo Android autenticou usuário QA, abrindo a tela inicial.

Tabela 28. Caso de teste de integração 1

A Tabela 29 descreve o teste de integração relacionado à solicitação de corrida com cálculo de rota. O teste avalia a interação entre o Mobile App, a MobU API e a API de Mapas, assegurando que origem e destino sejam processados corretamente e que o passageiro visualize estimativas de percurso e valor.

Identificador	TI2 – Solicitação de corrida com rota
Sistemas Envolvidos	Mobile App (Passageiro), MobU API e API de Mapas/Geo
Interface	HTTP REST entre App e MobU API (POST /rides/estimate e POST /rides); integração com serviço de mapas/geo para estimativa de distância, tempo e valor.
Pré-condição	Passageiro autenticado; localização de origem habilitada no dispositivo.
Entradas	1. Informar origem e destino no aplicativo/API dentro da região ativa. 2. Enviar estimativa (POST /rides/estimate). 3. Criar corrida (POST /rides) com método de pagamento selecionado.
Resultados	Resultado esperado: API de Mapas/Geo deve retornar rota e distância; MobU API deve calcular valor estimado e tempo de viagem e responder ao app com os dados consolidados. Resultado obtido: aprovado. Estimativa e criação de corrida foram processadas com coordenadas válidas de Pará de Minas, retornando valor, distância e registro da corrida.

Tabela 29. Caso de teste de integração 2

A Tabela 30 apresenta o caso de teste que verifica o envio de notificações push ao motorista quando uma nova corrida é confirmada. Esse teste garante o funcionamento integrado entre a MobU API, o serviço FCM e o aplicativo do motorista.

Identificador	TI3 – Notificação de nova corrida
Sistemas Envolvidos	MobU API, serviço de notificações push (FCM) e Mobile App (Motorista)
Interface	HTTP REST para registro do token (PATCH /auth/fcm-token); HTTP/HTTPS entre MobU API e FCM; canal de Push Notification entre FCM e App.
Pré-condição	Motorista autenticado e marcado como online ; token de notificação registrado na API.
Entradas	1. Passageiro confirma solicitação de corrida. 2. MobU API envia mensagem de nova corrida ao FCM com dados resumidos (origem, destino, valor estimado).
Resultados	Resultado esperado: motorista deve receber uma notificação push no dispositivo e abrir a tela de detalhes da corrida ao tocar na notificação. Resultado obtido: parcial. O token FCM foi registrado e persistido, e o backend acionou o fluxo de notificação; o recebimento visual da push notification não foi comprovado em emulador local por depender da infraestrutura externa do Firebase.

Tabela 30. Caso de teste de integração 3

A Tabela 31 descreve o teste de integração destinado a validar o processo de registro de pagamento PIX offline, sem API bancária. O teste garante que a confirmação enviada pelo motorista seja corretamente recebida e registrada pela API.

Identificador	TI4 – Registro de pagamento PIX off-line
Sistemas Envolvidos	Mobile App (Passageiro/Motorista) e MobU API
Interface	HTTP REST (POST /payments/rides/{rideId}/confirm, POST /payments/rides/{rideId}/not-received e POST /payments/confirm para compatibilidade).
Pré-condição	Corrida finalizada e QR Code PIX já gerado pela API; passageiro realizou o pagamento utilizando o aplicativo bancário externo.
Entradas	1. Motorista informa, no app, que o pagamento foi recebido. 2. App envia confirmação de pagamento à API com o identificador da corrida.

Resultados	Resultado esperado: MobU API deve atualizar o status do pagamento e refletir o novo estado no histórico de corridas do passageiro e do motorista. Resultado obtido: aprovado. A API registrou pagamento não recebido, confirmou recebimento posteriormente e preservou txId/payload Pix nos detalhes da corrida.
-------------------	--

Tabela 31. Caso de teste de integração 4

A Tabela 32 apresenta o teste de integração sobre a atualização do status do motorista entre online e offline. O teste assegura que o aplicativo acione a API corretamente e que o estado atualizado seja refletido no sistema.

Identificador	TI5 – Alternar status online/off-line
Sistemas Envolvidos	Mobile App (Motorista) e MobU API
Interface	Requisição HTTP REST (POST /driver/status).
Pré-condição	Motorista autenticado no aplicativo.
Entradas	1. Motorista aciona o botão “Ficar online/off-line” na tela principal do app.
Resultados	Resultado esperado: API deve atualizar o status do motorista no banco de dados; o aplicativo deve receber confirmação e exibir o novo estado. Resultado obtido: aprovado. O status online/off-line foi atualizado pela rota atual do backend e passou a impactar a busca/listagem de corridas pendentes.

Tabela 32. Caso de teste de integração 5

A Tabela 33 descreve o teste que valida o fluxo de aceitação de corrida pelo motorista. O caso assegura que a API atualize o estado da corrida e que a API de Mapas forneça o trajeto adequado até o local de embarque.

Identificador	TI6 – Aceitar corrida e iniciar rota
Sistemas Envolvidos	Mobile App (Motorista), MobU API e API de Mapas/Geo
Interface	HTTP REST entre App e MobU API (GET /driver/rides/pending e POST /driver/rides/{rideId}/accept); integração com mapas/geo para rota de deslocamento.

Pré-condição	Corrida em estado DESPACHADA para o motorista; motorista em modo online .
Entradas	1. Motorista consulta corridas pendentes. 2. Motorista clica em “Aceitar corrida”. 3. Aplicativo/API atualizam a corrida e solicitam dados de rota.
Resultados	Resultado esperado: MobU API atualiza a corrida para estado ACEITA; API de Mapas/Geo retorna a rota até a origem; o aplicativo exibe o trajeto no mapa. Resultado obtido: aprovado. A corrida pendente foi listada para o motorista online, aceita via API e vinculada corretamente ao motorista.

Tabela 33. Caso de teste de integração 6

A Tabela 34 apresenta o teste de integração que avalia o processo de finalização de corrida com pagamento em dinheiro. O teste confirma que a API atualiza o estado da corrida e registra o pagamento corretamente.

Identificador	TI7 – Finalizar corrida com pagamento em dinheiro
Sistemas Envolvidos	Mobile App (Motorista/Passageiro) e MobU API
Interface	Requisição HTTP REST (PATCH /driver/rides/{rideId}/arriving, /arrived, /start e /finish).
Pré-condição	Corrida em estado EM_ANDAMENTO ; método de pagamento configurado como DINHEIRO .
Entradas	1. Motorista avança os estados da corrida até o destino. 2. App envia para a API o identificador da corrida e confirma a finalização.
Resultados	Resultado esperado: MobU API deve atualizar o estado da corrida para FINALIZADA/FINISHED e registrar o pagamento conforme o método selecionado; histórico de passageiro e motorista deve exibir a corrida. Resultado obtido: aprovado. O fluxo de progressão e finalização foi concluído e os históricos de motorista e passageiro refletiram a corrida finalizada.

Tabela 34. Caso de teste de integração 7

A Tabela 35 descreve o caso de teste utilizado para validar a consulta de histórico de ganhos e corridas realizadas. O teste garante o funcionamento adequado da API ao retornar dados consolidados com base nos filtros selecionados pelo motorista.

Identificador	TI8 – Consultar histórico de corridas e ganhos
Sistemas Envolvidos	Mobile App (Motorista) e MobU API
Interface	Requisição HTTP REST (GET /driver/rides/history).
Pré-condição	Motorista autenticado e com corridas finalizadas registradas no sistema.
Entradas	1. Motorista acessa a tela “Histórico e ganhos” e seleciona um período (ex.: última semana).
Resultados	Resultado esperado: MobU API deve retornar lista de corridas do período, valores individuais e total acumulado; o aplicativo deve exibir o extrato de ganhos. Resultado obtido: aprovado. A rota atual de histórico retornou corridas finalizadas e dados financeiros utilizados pelas telas de histórico/ganhos do motorista.

Tabela 35. Caso de teste de integração 8

A Tabela 36 apresenta o teste de integração referente ao módulo administrativo do sistema, verificando a geração de relatórios por meio de consultas agregadas realizadas pela API. O teste comprova que filtros selecionados pelo administrador resultam em indicadores consistentes para gestão.

Identificador	TI9 – Relatórios administrativos
Sistemas Envolvidos	Painel Web/Admin (ou módulo futuro), MobU API e SGBD
Interface	HTTP REST entre Painel Web/Admin e MobU API (GET /admin/reports, GET /admin/passengers, GET /admin/drivers e PATCH /admin/users/{id}/block); conexão SQL entre API e banco de dados.
Pré-condição	Usuário administrador autenticado; dados de corridas, pagamentos e usuários previamente cadastrados.
Entradas	1. Administrador autentica no painel. 2. Painel consulta usuários/relatórios e aplica filtros administrativos.

	3. Painei envia ação de bloqueio quando necessário.
Resultados	Resultado esperado: MobU API executa consultas agregadas e retorna indicadores para o painel administrativo. Resultado obtido: aprovado. Relatórios, listas de usuários e bloqueio administrativo foram validados contra a API e banco de dados local.

Tabela 36. Caso de teste de integração 9

7. Cronograma e Processo de Implementação

Nesta seção é apresentado o cronograma de desenvolvimento do sistema MobU ao longo do primeiro semestre de 2026. O cronograma está organizado por quinzenas, iniciando na primeira quinzena de fevereiro de 2026 e encerrando na segunda quinzena de junho de 2026. Em cada período são listadas as atividades a serem desenvolvidas, contemplando a construção da aplicação móvel, API, infraestrutura, testes, documentação e validações finais.

7.1 Cronograma

O cronograma do projeto MobU foi estruturado em períodos quinzenais ao longo do primeiro semestre de 2026, permitindo uma organização clara das entregas e um acompanhamento contínuo da evolução do desenvolvimento. Cada quinzena contempla um conjunto de atividades planejadas, abrangendo desde a preparação inicial dos ambientes, definição da arquitetura e modelos de dados, até a implementação progressiva dos fluxos da aplicação móvel e da API, testes, ajustes e validações.

A divisão quinzenal possibilita que o projeto seja desenvolvido de forma incremental, garantindo que funcionalidades essenciais sejam entregues gradualmente e validadas ao longo do processo. Dessa forma, cada período apresenta objetivos específicos que contribuem diretamente para a construção completa do sistema, incluindo desenvolvimento do aplicativo móvel, camada de serviços, integração com APIs externas, persistência de dados, testes de aceitação e integração, além de atividades de documentação e refinamentos finais.

A Tabela a seguir apresenta a distribuição das atividades planejadas para cada quinzena, detalhando o escopo de trabalho previsto para todo o ciclo de desenvolvimento do MobU.

Período	Atividades
01/02 – 15/02 (1ª Quinzena de Fevereiro)	• Configuração inicial dos repositórios (Mobile + API).

	● Definição da arquitetura geral do sistema (camadas, padrões e tecnologias). ● Criação da estrutura de pastas do aplicativo móvel: ○ Estrutura base ○ Tema da aplicação ○ Instalação de bibliotecas essenciais ○ Arquivo de rotas e navegação ● Criação da estrutura inicial da API (Node/Express): ○ Estrutura de pastas ○ Instalação de dependências principais ○ Configuração de ambiente e variáveis (.env) ● Configuração inicial do banco de dados PostgreSQL. ● Criação do primeiro DER rascunho.
16/02 – 29/02 (2ª Quinzena de Fevereiro)	● Implementação da tela de login e fluxo de autenticação no Mobile. ● Implementação dos casos de uso: ○ UC01 Criar conta / Entrar ○ UC02 Gerenciar Perfil ● Implementação dos primeiros endpoints de autenticação da API.

	● Integração mobile ↔ API (login, criação de usuário). ● Implementação da estrutura de tokens (JWT). ● Configuração do ambiente Docker da API.
01/03 – 15/03 (1ª Quinzena de Março)	● Implementação dos casos de uso relacionados às corridas: ○ UC03 Informar origem/destino ○ UC04 Estimar tarifa ○ UC05 Solicitar corrida ● Comunicação com API externa de mapas/geo (cálculo de rota e distância). ● Implementação dos endpoints de cálculo de tarifa e solicitação de corrida. ● Modelagem final das entidades: ○ Usuário ○ Motorista ○ Veículo ○ Corrida ○ Localização
16/03 – 31/03 (2ª Quinzena de Março)	● Implementação das telas do motorista: ○ UC10 Ficar online/offline ○ UC11 Receber solicitações

	○ UC12 Aceitar/recusar corrida ● Implementação do fluxo de despacho de corridas. ● Implementação de notificações via FCM (entrada e aceite de corrida). ● Implementação dos endpoints: ○ Despacho ○ Aceite ○ Rejeição ● Testes de integração Mobile ↔ API ↔ FCM.
01/04 – 15/04 (1ª Quinzena de Abril)	● Implementação do fluxo operacional da corrida: ○ UC13 Navegar até embarque/destino ○ UC14 Marcar etapas (cheguei/iniciei/finalizei) ● Implementação de trilhas de localização e mapas no aplicativo do motorista. ● Configuração final do sistema de tracking em tempo real. ● Implementação de testes de aceitação para todo o fluxo até finalização da corrida.

16/04 – 30/04 (2ª Quinzena de Abril)	● Implementação do sistema de pagamento: ○ UC08 Pagar (PIX/Dinheiro) ● Geração do QR Code PIX dentro da aplicação (offline, sem PSP). ● Implementação dos estados de pagamento (Figura XX): ○ Aguardando ○ PIX_GERADO ○ Aguardando confirmação ○ Confirmado / Cancelado ● Implementação dos endpoints de pagamento e confirmação do motorista. ● Implementação das telas e fluxos relacionados. ● Testes de integração Mobile ↔ API ↔ Pagamentos.
01/05 – 15/05 (1ª Quinzena de Maio)	● Implementação do histórico de corridas: ○ UC07 Cancelar corrida ○ UC09 Avaliar motorista ○ UC15 Ver histórico e ganhos ● Implementação dos relatórios internos (admin): ○ UC17 Ver relatórios e métricas ○ UC18 Configurar taxas e regiões

	● Criação de Jobs assíncronos (limpeza, cálculos, fechamentos). ● Testes de integração ampliados.
16/05 – 31/05 (2ª Quinzena de Maio)	● Desenvolvimento do painel administrativo (versão web simples opcional). ● Finalização da documentação técnica: ○ Diagramas UML ○ DER final ○ Diagramas de componentes e implantação ● Finalização do módulo de notificações e eventos. ● Testes de carga e otimização de consultas. ● Correções gerais com base nos testes das sprints anteriores.
01/06 – 15/06 (1ª Quinzena de Junho)	● Configuração de pipelines CI/CD. ● Publicação do aplicativo em ambiente de testes. ● Homologação interna com cenários completos.
16/06 – 30/06 (2ª Quinzena de Junho)	● Ajustes finais após homologação. ● Últimos testes de aceitação. ● Preparação e entrega da documentação do projeto. ● Apresentação e encerramento do semestre.

	• Pós-mortem do projeto.
--	--

Tabela 16. Cronograma de Implementação